

Skytree Python SDK

Release 15.4.0

SKYTREE®

Notices

Copyright

Copyright 2014-2016 Skytree, Inc. All rights reserved.

Information in this document is furnished only under a Customer License Agreement or non-disclosure agreement and may be used or copied only in accordance with the terms of either such agreement. The software described in this document is owned by Skytree, Inc. and protected by U.S. copyright law and may not be copied on any medium except as specifically authorized in the Customer License Agreement or non-disclosure agreement.

Disclaimer

Skytree, Inc. reserves the right to revise this document and to make changes from time to time without any obligation of any kind to notify any person of such revisions or changes.

Trademarks

SKYTREE®, the Skytree logo, THE MACHINE LEARNING COMPANY®, THE DATA DRIVEN ENTERPRISE™, Skytree Infinity™, AutoModel™, and AutoDocumentation™ are trademarks or registered trademarks of Skytree, Inc. ORACLE®, JAVA®, and JDK™ are trademarks or registered trademarks of Oracle and/or its affiliates. HADOOP®, HIVE™, PIG™, HBASE™, SPARK™, TOMCAT™, and ZOOKEEPER™ are trademarks or registered trademarks of the Apache Software Foundation. MAPR® is a registered trademark of MapR Technologies, Inc. Python™ is a trademark of the Python Software Foundation. Third-party product names and related marks used in this document are trademarks or registered trademarks of their respective owners. Use of third-party product names and marks does not imply any affiliation or endorsement by such parties.

Company Information:

Skytree, Inc.
1731 Technology Drive, Suite 700
San Jose, CA 95110
408.392.9300
www.skytree.net

Customers with a support contact can contact support using any of the following methods:

- Log in to support.skytree.net.
- Send questions to support@skytree.net.
- Call 408.392.9300 and select option 3.
- You can also view additional Skytree documentation by logging on to support.skytree.net.

Freemium customers can send inquiries to the LinkedIn® Community Group for Skytree:

<https://www.linkedin.com/groups/4468542>

Revision Date: March 03, 2016

Contents

Chapter 1 About this Document 1

Assumptions 1

Document Convention 2

Chapter 2 Installation and Import 3

Requirements 3

Installation 3

Chapter 3 The skytree Module 5

Project Instance Methods 5

Dataset Instance Methods 6

Project Properties 7

Dataset Properties 8

Custom Transform Properties 9

skytree.authenticate()10

skytree.create_project() 11

skytree.list_projects()12

skytree.get_project()13

skytree.get_project_dependents()14

skytree.delete_project()15

skytree.list_sources()16

skytree.get_source()17

skytree.get_source_dependents()18

skytree.delete_source()19

skytree.add_custom_transform()20

skytree.list_custom_transforms() 21

skytree.get_custom_transform() 22

project.create_dataset()23

project.list_datasets()26

project.list_models()27

project.list_results()28

project.list_plots()29

project.get_dataset()30

project.get_dataset_depenedents() 30

project.get_model()31

project.get_model_dependents32

project.get_result()33

project.get_result_dependents()34

project.get_plot()35

project.get_plot_dependents()	36
project.delete_dataset()	37
project.delete_model()	38
project.delete_result()	39
project.delete_plot()	40
project.rename()	41
dataset.get_log()	42
dataset.terminate()	43
dataset.commit()	44
dataset.sample()	45
dataset.summary()	46
dataset.add_column()	49
dataset.add_unique_id_column()	50
dataset.clamp_values()	51
dataset.extract_text_features()	53
dataset.filter()	54
dataset.horizontalize()	55
dataset.ingest_text()	57
dataset.ingest_url()	58
dataset.inner_join()	59
dataset.left_outer_join()	60
dataset.normalize(), dataset.normalize_unit(), dataset.normalize_standard()	61
dataset.remove_columns()	63
dataset.right_outer_join()	64
dataset.set_as_categorical_number()	65
dataset.set_as_categorical_text()	66
dataset.set_as_id()	68
dataset.split()	69
dataset.tokenize_text	71
dataset.vectorize_text()	72
dataset.apply_transform_steps()	73
Dataset.apply_custom_transform()	75

Chapter 4 The skytree.prediction Module 77

Model Properties	78
Test Result Properties	79
Classification Score Configuration Properties	81
skytree.prediction.learn()	82
model.terminate()	84
model.summary()	85
model.get_log()	87
model.get_pdp_variable_names()	88
model.get_pmml_gzip()	90
PMML Usage Restrictions	90
models.visualize_pdp()	92
model.test()	93

PlotRegistry()	95
plot_registry_entity.findall()	96
plot_registry_entity.findById()	97
plot.terminate()	98
test.create_probabilities_dataset()	99
test.create_labels_dataset()	100
test.create_targets_dataset()	102
test.summary()	103
test.terminate()	104
test.visualize()	105
test.get_log()	108
test.get_probabilities()	109
test.get_labels()	110
test.get_targets()	111

Appendix A Models 113

Classification vs. Regression	113
Methods	113
AutoModel Models	114
GBT Models	114
GBTR Models	114
Generalized Linear Models - Classification and Regression	114
RDF Models	115
RDFR Models	115
SVM Models	116
Model Configuration	117
AutoModel Configuration	118
GBT Model Configuration	119
GBTR Model Configuration	124
GLMC Model Configuration	128
GLMR Model Configuration	131
RDF Model Configuration	134
RDFR Model Configuration	137
SVM Model Configuration	139
Model Examples	142
AutoModel Examples	142
A Simple Regression Example	142
GBT Model Examples	142
Tuning the Number of Trees with a Holdout Set	142
Tuning with Smart Search	143
GBTR Model Examples	143
Tuning with a Loss Function	143
Tuning the Number of Trees using K-Fold Cross Validation	144
GLM Model Examples	144
Training a Gaussian-Identity Model	144
Excluding the Bias Term	145

Tuning an Unbiased Poisson-Log Model	145
RDF Model Examples	146
Tuning the Number of Dimensions	146
Tuning for Skewed Data Automatically	146
RDFR Model Examples	147
Tuning for the Sampling Ratio	147
Tuning the Minimum Node Size	147
SVM Model Examples	147
Tuning a Non-Linear SVM	148
Tuning for Best Accuracy	148

Appendix B Custom Script Objects 149

sc	149
Input	150
Output	151
Column	151
RDD	151
Double RDD	154
Pair RDD	155

Index 161

Chapter 1 About this Document

This document is intended for users who prefer to utilize the Python SDK rather than the curl commands of the REST API to use Skytree and provides examples for performing the following functions:

- Create projects
- Create and transform datasets
- Train datasets to create models
- Test and score models
- Make predictions
- Generate plots for models and results

Assumptions

- This document assumes that you know the location of the Skytree Python SDK directory. This information must come from the Admin who installed the Skytree software.
- This document assumes that the Admin has created a user account for you and provided you with authentication requirements.
- This document assumes that you are familiar with Skytree methods. Information about the available methods is included in the *Skytree Server User Guide*, which describes the method used for configuring Skytree.
- Setuptools must already be bootstrapped. The Python SDK cannot install without setuptools. Any version compatible with Python 2.7 is supported. At the time of this writing, the current setuptools version was 15.2. Refer to the python.org site for installation instructions.
- This document includes code samples that users can copy and paste. This method works best when the document is viewed in Adobe® Reader®.
- In order to successfully run the examples included in this document, we assume that the following files are available in the <skytree_api_python_sdk>/examples directory.
 - `income.data` - for Classification examples
 - `income.test` - for Model Testing examples
 - `matrix.csv` - for Regressions examples
 - `join1.csv` - for Join Transform examples

- `join2.csv` for Join Transform examples

Document Convention

Function and attribute descriptions in this document use the following conventions:

Convention	Description
<code>monospace</code>	Monospace font indicates a function or attribute that should be entered as it appears, such as functions, attributes, and keywords
<i>italicized sans serif</i>	Italicized sans serif font indicates an object of a method instance or class. These represent values that a user sets and then later calls. For example: <pre><i>project_object</i>.create_dataset()</pre> In the above function, a user defines a value for the project object. This object is then referenced in later method instances of the project class.
<i>italicized serif</i>	Italicized serif font indicates arguments for a method or attribute that the user supplies. For example: <pre>name=<i>new_name</i></pre> In the above example, a user provides a value for <i>new_name</i>.

Chapter 2 Installation and Import

This section describes how to set up your environment to use the Python SDK.

Requirements

The Skytree Python SDK requires Python 2.7. The only mandatory third-party dependencies are the following libraries, which are automatically installed during the setup:

- requests <<https://github.com/kennethreitz/requests>>
- poster <<http://atlee.ca/software/poster/#download>>
- enum34 <<https://pypi.python.org/pypi/enum34>>

Installation

Perform the following steps to install/upgrade Python:

```
$ cd <api_directory>
$ python setup.py install
```

Note that the above command assumes that you have sudo access on the machine. If you do not, then run the following instead:

```
$ python setup.py install --user
```

The Skytree Python SDK consists of the following modules:

- [The skytree Module](#) (page 5)
- [The skytree.prediction Module](#) (page 77)

Run the following commands in Python to import these modules and the related model classes.

```
$ python
>>> import skytree
>>> from skytree import CustomTransform, DatasetConfig, TransformSteps
>>> import skytree.prediction
>>> from skytree.prediction import automodel, gbt, gbtr, glmc, glmr, rdf, rdfr, svm
>>> from skytree.prediction import ClassificationScoreConfig
>>> from skytree.prediction import Plot, PlotRegistry
```


Chapter 3 The skytree Module

The **skytree** module includes functions for authenticating, for creating and retrieving project information, for retrieving source information, and adding custom transforms.

- `skytree.authenticate()`
- `skytree.create_project()`
- `skytree.list_projects()`
- `skytree.get_project()`
- `skytree.get_project_dependents()`
- `skytree.delete_project()`
- `skytree.list_sources()`
- `skytree.get_source()`
- `skytree.get_source_dependents()`
- `skytree.delete_source()`
- `skytree.add_custom_transform()`
- `skytree.list_custom_transforms()`
- `skytree.get_custom_transform()`

Project Instance Methods

The following instance methods of the Project class are used for working with projects. In addition, after a project is created, a variety of attributes become available. Refer to [Project Properties](#) (page 7) for more information.

- `project.create_dataset()`
- `project.list_datasets()`
- `project.list_models()`
- `project.list_results()`
- `project.list_plots()`
- `project.get_dataset()`

- `project.get_dataset_dependents()`
- `project.get_model()`
- `project.get_model_dependents()`
- `project.get_result()`
- `project.get_result_dependents()`
- `project.get_plot()`
- `project.get_plot_dependents()`
- `project.delete_dataset()`
- `project.delete_model()`
- `project.delete_result()`
- `project.delete_plot()`
- `project.rename()`

Dataset Instance Methods

The following instance methods of the Dataset class are used for working with datasets. In addition, after datasets are added to a project, a variety of attributes become available. Refer to [Dataset Properties](#) (page 8) for more information.

- `dataset.get_log()`
- `dataset.terminate()`
- `dataset.commit()`
- `dataset.sample()`
- `dataset.summary()`

In addition to the above, all transformations are Dataset instance methods.

Note: Only datasets with a READY status can be transformed. During transformations, columns that include only missing values will be set a categorical type "text" and will have an empty dictionary. The transformation will then complete with an error indicating that a dictionary is not available for the column and that continuing any machine learning with the dataset is not possible.

- `dataset.add_column()`
- `dataset.add_unique_id_column()`
- `dataset.clamp_values()`
- `dataset.extract_text_features()`
- `dataset.filter()`
- `dataset.horizontalize()`
- `dataset.ingest_text()`

- `dataset.ingest_url()`
- `dataset.inner_join()`
- `dataset.left_outer_join()`
- `dataset.normalize()`, `dataset.normalize_unit()`, `dataset.normalize_standard()`
- `dataset.remove_columns()`
- `dataset.right_outer_join()`
- `dataset.set_as_categorical_number()`
- `dataset.set_as_categorical_text()`
- `dataset.set_as_id()`
- `dataset.split()`
- `dataset.tokenize_text()`
- `dataset.vectorize_text()`

Multiple transforms can be combined using the `apply_transform_steps()` function. This function requires a `TransformSteps()` object, which defines the set of transforms to apply to a dataset.

- `dataset.apply_transform_steps()`

Skytree also allows you to create custom transformations using the `Dataset.apply_custom_transform()` function. This function requires a `Script()` object, which defines the Python script that will be referenced in the transform. After custom transformed datasets are added to a project, a variety of attributes become available. Refer to [Custom Transform Properties](#) (page 9) for more information.

- `Dataset.apply_custom_transform()`

Note: Due to constraints with Spark, you cannot chain more than 200 total operations when transforming a dataset. This constraint applies to the entire life cycle of a dataset (i.e., from initial dataset, through multiple transforms, to final dataset).

Finally, a `ready()` function should be included with every input request. The `ready()` function is a blocking call that polls the server continuously until the resource is available for use (i.e., has a READY status). Models, tests, and datasets must be READY before they can be used.

Project Properties

After a project is created, the following properties can be used to retrieve specific information.

Table 1: Project Properties

Property	Description
<code>project.id</code>	Shows the project ID.
<code>project.name</code>	Shows the project name.

Table 1: Project Properties (continued)

Property	Description
<code>project.description</code>	Shows the project description, if one was included when the project was created.
<code>project.created_at</code>	Shows the epoch time when the project was created.
<code>project.updated_at</code>	Shows the epoch time when the project was last updated.

Dataset Properties

After a dataset is successfully imported into a project, the following properties can be used to retrieve specific information.

Table 2: Dataset Properties

Property	Description
<code>dataset.id</code>	Shows the dataset ID.
<code>dataset.name</code>	Shows the dataset name.
<code>dataset.status</code>	Shows the dataset's status (for example, READY).
<code>dataset.number_of_rows</code>	Shows the number of rows in the dataset
<code>dataset.columns</code> or <code>dataset.columns[<i>i</i>]</code>	Use this without a variable to print the dataset features for all columns. Use this with a variable to print the features for a specific column.
<code>dataset.config</code> print <code>dataset.config</code>	Use this attribute to create a configuration instance for a specified dataset. Once created, you can use <code>print dataset.config</code> to view the configuration that was used when importing this dataset. This corresponds to the DatasetConfig object from the project.create_dataset() function and shows the ID column and blob columns (free text), if configured.
<code>dataset.state</code>	Shows the dataset's state. If the dataset was committed or used to create a model, then the state will be FULL. Otherwise, the state will be SAMPLED.
<code>dataset.type</code>	Shows whether the dataset was imported into the project or added via a transform.
<code>dataset.source_id</code>	If this dataset was generated from a source file, then this shows the source ID.
<code>dataset.transform_steps</code>	Shows the transform steps (if any) that were used to create this dataset.
<code>dataset.rest_api_response_dict</code>	Shows the dictionary entries for each column in the dataset.
<code>dataset.created_at</code>	Shows when the specified dataset was created, including the month, date, year, hour, minutes, and seconds.
<code>dataset.updated_at</code>	Shows when the specified dataset was last updated, including the month, date, year, hour, minutes, and seconds.

Custom Transform Properties

After a custom transformed dataset is successfully imported into a project, the following properties can be used to retrieve specific information.

Table 3: Custom Transform Properties

Property	Description
<code>custom_transform.id</code>	Shows the custom transformed dataset ID.
<code>custom_transform.name</code>	Shows the custom transformed dataset name.
<code>custom_transform.status</code>	Shows the custom transformed dataset's status (for example, READY).
<code>custom_transform.created_at</code>	Shows when the specified custom transformed dataset was created, including the month, date, year, hour, minutes, and seconds.
<code>custom_transform.rest_api_response_dict</code>	Shows the dictionary entries for each column in the custom transformed dataset.

skytree.authenticate()

All users must authenticate in order to access Skytree. A typical authentication process is as follows:

1. The Admin authorizes a new user by providing the user's email address and a password.
2. The Admin delivers the username (login name when using LDAP or e-mail address when not using LDAP), password, and hostname to the new user. Note that a valid password is a minimum of eight characters and contains at least one uppercase character, one lowercase character, and one number. Special characters are allowed but not required.
3. Using the information provided by the Admin, the user will include this information in the `authenticate` function to log in to Skytree.

```
skytree.authenticate(  
    "myUserName",  
    "myPassword1",  
    hostname="http://company.com:port/v1"  
)
```

Note: "http/https" is required in the hostname. Contact your Admin for the exact server location.

skytree.create_project()

A project contains the datasets, models, and tests results corresponding to a machine learning task. Each project retains a full audit train of how each dataset, model, or result was created.

The `skytree.create_project` function creates a project.

Usage

```
project_object = skytree.create_project(project_name, project_description)
```

Parameters

- *project_object* - Specify a variable to contain the new project
- *project_name* - A string containing the name of the project. No other project with this name should exist.
- *project_description* - Optionally add a description that provides more information about this project. The description is enclosed passed as a string.

Examples

Example 1

The example below creates a project named Customer Acquisition and assigns it the variable `ca`. This project will include, for example, datasets and models geared toward acquiring additional customers. This project will be referenced when creating a dataset. (See [project.create_dataset\(\)](#) (page 23).)

```
ca = skytree.create_project(  
    "Customer Acquisition",  
    "Acquire new customers"  
)
```

Example 2

The example below creates a project called `fraud`. This project will include, for example, datasets and models geared toward determining fraudulent activity.

```
fraud = skytree.create_project(  
    "Fraud Detection",  
    "Detecting fraudulent activity"  
)
```

Example 3

The example below creates a project called `predict`. In this example, the optional description is not provided.

```
predict = skytree.create_project("Prediction")
```

skytree.list_projects()

This function retrieves a list of all projects associated with the current user. This list can be printed after it is retrieved.

Usage

```
list_of_projects = skytree.list_projects()
```

Parameters

- *list_of_projects* - A variable that will contain the returned list of project objects

Example

The example below requests a list of all projects. Note that no additional parameters are required.

```
projects = skytree.list_projects()
```

Note that with Python, you can print details about projects. The example below retrieves information about all of the available projects.

```
for project in projects:  
    print project
```

This prints a list of each project ID and name. Note that this project ID is required in the [get_project\(\)](#) function. An example output is below:

```
name=Prediction, id=2628892886694058262, description=None  
name=Fraud Detection, id=392252979972263403, description=Detecting  
    fraudulent activity  
name=Customer Acquisition, id=5151422772595357676, description=  
    Acquiring new customers
```

skytree.get_project()

Use this function to retrieve a specific project by its ID. This project can then be used to create datasets, models, and results. It is possible to have multiple projects, but each dataset, model, and result can only be associated with one project.

Usage

```
project_object = skytree.get_project(project_id)
```

Parameters

- *project_object* - Specify a variable that will contain the retrieved project object
- *project_id* - A string containing the unique identifier for the project you want to retrieve. The identifier for a particular project can be found by printing it. All projects can be retrieved with the [skytree.list_projects\(\)](#) function.

Example

The example below gets the three projects created in the [skytree.create_project\(\)](#) example.

```
predict = skytree.get_project("2628892886694058262")
fraud = skytree.get_project("392252979972263403")
ca = skytree.get_project("5151422772595357676")
```

skytree.get_project_dependents()

Use this function to retrieve a list of all datasets, models, results, and plots included in a project.

Usage

```
project_object = skytree.get_project_dependents(project_id)
```

Parameters

- *project_object* - Specify a variable that will contain the retrieved project object
- *project_id* - A string containing the unique identifier for the project you want to retrieve. The identifier for a particular project can be found by printing it. All projects can be retrieved with the [skytree.list_projects\(\)](#) function.

Example

The example below retrieves all files in the predict (id of 2628892886694058262) project.

```
project = skytree.get_project_dependents("2628892886694058262")
```

The following example prints the files that are in this project:

```
>>> for project in projects:
...     print project
...
Id:4520433320984416731, name=Prediction, type=PROJECT, is_deletable=True
Id:4274348704094814824, name=income.data, type=DATASET, is_deletable=True
Id:4426696063906522815, name=income.data.ids, type=DATASET, is_deletable=True
Id:733579628360679490, name=GBT Result, type=TESTRESULT, is_deletable=True
Id:8613266560949776288, name=GBT Model, type=MODEL, is_deletable=True
Id:5350585628603483618, name=GBTR Model, type=MODEL, is_deletable=True
```

skytree.delete_project()

Use this function to delete a project. All associated files (datasets, models, tests) will also be deleted as a result. We recommend that you review the list of resources included in a project before deleting the project. Refer to [skytree.get_project_dependents\(\)](#) for more information.

Usage

```
skytree.delete_project(project_id)
```

Parameters

- *project_id* - A string containing the unique identifier for the project you want to retrieve. The identifier for a particular project can be found by printing it. All projects can be retrieved with the [skytree.list_projects\(\)](#) function.

Examples

The following example deletes a project whose ID is 6183292511729895174.

```
skytree.delete_project('6183292511729895174')
```

skytree.list_sources()

This function retrieves a list of all projects associated with the current user. This list can be printed after it is retrieved.

Usage

```
list_of_projects = skytree.list_projects()
```

Parameters

- *list_of_projects* - A variable that will contain the returned list of project objects

Example

The example below requests a list of sources. Note that no additional parameters are required.

```
listofsources = skytree.list_sources()
```

With Python, you can print details about sources. The example below retrieves information about all of the available sources.

```
for source in listofsources:  
    print source.id  
    print source.status
```

This prints the iD and status of all available sources. An example output is below:

```
created_at=2016-02-28 13:47:41, id=1476101491786137726, url=imported,  
status={u'message': u'Source is ready to be used', u'code': u'READY'}  
created_at=2016-02-28 13:46:49, id=4934400838214191390, url=imported,  
status={u'message': u'Source is ready to be used', u'code': u'READY'}  
created_at=2016-02-28 13:42:49, id=1694799706453673076, url=imported,  
status={u'message': u'Source is ready to be used', u'code': u'READY'}  
created_at=2016-02-27 16:00:20, id=8643411948654686562, url=imported,  
status={u'message': u'Source is ready to be used', u'code': u'READY'}
```

skytree.get_source()

Use this function to retrieve a specific source by its ID.

Usage

```
source_object = skytree.get_source(source_id)
```

Parameters

- *source_object* - Specify a variable that will contain the retrieved source object
- *source_id* - A string containing the unique identifier for the source you want to retrieve. The identifier for a particular source can be found using the [skytree.list_sources\(\)](#) function.

Example

The example below retrieves a source whose ID is 1694799706453673076.

```
source = skytree.get_source("1694799706453673076")
```

skytree.get_source_dependents()

Source files are created when running a test result. Use this function to get a list of all downstream (dependent) files related this source.

Usage

```
source_object = skytree.get_source_dependents(source_id)
```

Parameters

- *source_object* - Specify the variable that will contain the source object.
- *source_id* - A string containing the unique identifier for the source that you want to retrieve. IDs can be viewed by printing a list of sources or viewing its summary in the SDK.

Example

The example below retrieves all dependent files related to a source whose ID is 1694799706453673076.

```
sourcefiles = skytree.get_source_dependents("1694799706453673076")
```

The following example prints the list of files.

```
>>> for dependents in sourcefiles:  
...     print dependents  
...  
Id:1694799706453673076, name=income.test.ids, type=SOURCE, is_deletable=True
```


skytree.delete_source()

Use the `delete_source` function to delete a source file along with all downstream files that were generated from this dataset (for example, datasets, models, results, etc.). We recommend that you review all associated files before deleting a dataset. Refer to [skytree.get_source_dependents\(\)](#).

Note: You cannot delete application type source files (for example, source files that were built during a test result run and then used in that result).

Usage

```
skytree.delete_source(source_id)
```

Parameters

- *source_id* - Specify the ID that you want to delete. You can obtain the source ID using the `list_sources()` function. (Refer to "[skytree.list_sources\(\)](#)" on page 16 for more information.)

Example

The following example deletes the `income.test.id` source file. The ID for this file is 1694799706453673076.

```
skytree.delete_dataset("1694799706453673076")
```

skytree.add_custom_transform()

This function adds a Python script to your workspace. Once added, the script is accessible across all projects.

Usage

```
custom_script_object = skytree.add_custom_transform(  
    script_name, script_path, [import_paths]  
)
```

Parameters

- *custom_script_object* - Specify a variable to contain the custom transform
- *script_name* - A string containing the name of the script that this custom transform will apply
- *script_path* - A string containing the path to the script
- *import_paths* - If the script includes references to other files that are to be imported with this script, then include those paths here. Otherwise, leave this as [].

Examples

The example below creates a custom transform that includes the `find_replace.py` script. This script specifies to replace text in a column. The exact text/string to be found and then replaced as well as the column number in which to search are specified later in the [Dataset.apply_custom_transform\(\)](#) function.

```
custom_transform = skytree.add_custom_transform(  
    "Replace Column",  
    "tests/find_replace.py",  
    []  
)
```

skytree.list_custom_transforms()

This function retrieves a list of all custom transform scripts associated with the current user. This list can be printed after it is retrieved.

Usage

```
list_of_custom_transforms = skytree.list_custom_transforms()
```

Parameters

- *list_of_custom_transforms* - A variable that will contain the returned list of custom transform objects

Example

The example below requests a list of all custom transform scripts. The list is then printed.

```
custom = skytree.list_custom_transforms()
for custom in skytree.list_custom_transforms():
    print custom
```

skytree.get_custom_transform()

Use this function to retrieve a specific custom transform by its ID.

Usage

```
custom_script_object = skytree.get_custom_transform(custom_script_id)
```

Parameters

- *custom_script_object* - Specify a variable that will contain the retrieved custom script
- *custom_script_id* - A string containing the unique identifier for the custom transform script that you want to retrieve. The identifier for a particular custom transform script can be found by printing it. All custom transform scripts can be retrieved with the [skytree.list_custom_transform\(\)](#) function.

Example

The example below gets the custom script created in the [skytree.add_custom_transform\(\)](#) example.

```
custom_transform = skytree.get_custom_transform("8255633521972868370")
```

`project.create_dataset()`

A dataset represents information in tabular form that has been imported into Skytree. Datasets can be created and viewed through the GUI or through the Python SDK. Currently, datasets support several types of columns: numerical, categorical, and unique ID columns.

This function creates a dataset from a data file.

Usage

Import Local File

```
dataset_object = project_object.create_dataset(  
    path=path,  
    name=name,  
    has_header=has_header,  
    separator=separator,  
    missing_value=missing_value,  
    config=DatasetConfig(dataset_configuration)  
) .ready()
```

Importing from HTTP/HDFS

```
dataset_object = project_object.create_dataset(  
    url=http://address or hdfs://address,  
    name=name,  
    has_header=has_header,  
    separator=separator,  
    missing_value=missing_value,  
    config=DatasetConfig(dataset_configuration)  
) .ready()
```

Parameters

- *dataset_object* - Specify a variable to contain the new dataset
- *project_object* - Specify the project object that will contain the dataset
- *path* - If importing a local file, specify a string for the path to the file. *path=path* cannot be used with *url=http://* or *url=hdfs://*
- *http://address* or *hdfs://address* - If importing a file from the Web or a file on a file system, specify a string for the path to the file. *url=http://* or *url=hdfs://* cannot be used with *path=path*.

Note: When specifying an HDFS path, if the path to the file that you want to register is an absolute path, then be sure to include the beginning "/" (for example, *hdfs:///path/to/file*).

- *name* - A string containing the name to assign to this dataset (optional).
- *has_header* - A Boolean specifying whether to treat the first line of the data file as a header with column names. Note that column names can be a maximum of 255 characters. If *has_header=False*, then the columns are given default names.
- *separator* - Specify the character used to separate values in the dataset, such as ","
- *missing_value* - Specify the character used in the file to indicate missing values, such as "?"

- *dataset_configuration* - Optionally add a DatasetConfig object, which allows you to specify an ID column for the dataset as well as a set of columns to consider as free text ('blob' columns). A DatasetConfig can be initialized with the following two parameters:
 - *id_column* - a string containing the name of a column to treat as an ID
 - *blob_columns* - a list of strings containing the names of columns to treat as free, non-categorical text (for example, a description or a review). A dictionary will not be built on these columns, and the column will display "..." in the dataset summary.

Examples

Example 1

This example adds a dataset to the ca project, which was created using [create_project\(\)](#). The file being imported is the `income.data.ids` file in the `examples` folder. The `income.data.ids` file includes a header row, its entries are separated by commas, and missing values are indicated with a `?`. Finally, the `id` column of the dataset is specified as the ID column.

```
dataset1 = ca.create_dataset(
    path="examples/income.data.ids",
    name="income.data",
    has_header=True,
    separator="," ,
    missing_value="?",
    config=DatasetConfig(id_column="id")
).ready()
```

Note: The config option above assumes that you imported DatasetConfig during the Python Binding stage. (Refer to [Installation and Import](#) (page 3).) If you did not, then the option will be as follows:
`config=skytree.DatasetConfig(id_column="id")`

Example 2

This example below adds the `matrix.csv` dataset to the Customer Acquisition (ca) project. This file does not have a header. In addition, an ID column is not specified. This can be added later using the [set_as_id\(\)](#) transform.

```
dataset2 = ca.create_dataset(
    path="examples/matrix.csv",
    name="matrix.data",
    has_header=False,
    separator="," ,
    missing_value="?"
).ready()
```

Example 3

This example below adds the `rawExtractFeatures` dataset to the Prediction (`predict`) project. This includes a header. An ID column is specified, as is a blob (text) column. This dataset will be referenced later in the [dataset.ingest_text](#) transform.

```
raw = predict.create_dataset(
    path="examples/extract_features.text",
    name="rawExtractFeatures",
    has_header=True,
    separator="|" ,
```

```
        config=DatasetConfig(id_column="id", blob_columns=["doc"])
    ).ready()
```

Example 4

This example below adds the `urlRaw` dataset to the Prediction (`predict`) project. This dataset includes a list of URLs that point to individual text files. This dataset will be referenced later in the [dataset.ingest_url](#) transform.

```
urlRaw = predict.create_dataset(
    path="examples/url_ingest.txt",
    name="rawURLs",
    has_header=True,
    separator="|",
    config=DatasetConfig(id_column="id", blob_columns=["doc"])
).ready()
```

`project.list_datasets()`

Use this function to retrieve the datasets that are included in a specific project.

Usage

```
list_of_datasets = project_object.list_datasets()
```

Parameters

- `list_of_datasets` - A variable that will contain the returned list of dataset objects
- `project_object` - The project object that includes the datasets that you want to retrieve

Example

The example below requests to view a list of the datasets available in the `ca` project. The list that will contain the returned datasets is specified as `datasets`.

```
datasets = ca.list_datasets()
for dataset in datasets:
    print dataset.name
    print dataset.id
    print dataset.status
```

This prints the dataset name, ID, and status of datasets in the specified project. An example output is below:

```
income.data
5434731090186968769
{'u'message': u'SAMPLED: Data Set has been validated and is ready
to use', u'code': u'READY'}
```


project.list_models()

Use this function to retrieve all models that are included in a specific project.

Usage

```
list_of_models = project_object.list_models()
```

Parameters

- *list_of_models* - A variable that will contain the returned list of model objects
- *project_object* - The project object that includes the models that you want to retrieve

Example

The example below requests to view all models available in the `predict` project.

```
models = predict.list_models()
for model in models:
    print model.name
    print model.id
```

This prints the name and ID for each model in the specified project object. An example output is below:

```
Education
8534304197005081735
Income Prediction
477270195472925546
```

`project.list_results()`

Use this function to retrieve all results that are included in a specific project.

Usage

```
list_of_results = project_object.list_results()
```

Parameters

- `list_of_results` - A variable that will contain the returned list of result objects
- `project_object` - The project object that includes the results that you want to retrieve

Example

The example below requests to view all test results available in the `predict` project.

```
result = predict.list_results()
for result in results:
    print result.name
    print result.id
```

This prints the name and ID of results in the specified project object. An example output is below:

```
Education Prediction
8443502907664023107
Yearly Income Prediction
2163781905757607220
```

`project.list_plots()`

Use this function to retrieve all plots that are included in a specific project.

Usage

```
list_of_plots = project_object.list_plots()
```

Parameters

- `list_of_plots` - A variable that will contain the returned list of plot objects
- `project_object` - The project object that includes the plots that you want to retrieve

Example

The example below requests to view all plots available in the `predict` project.

```
plots = predict.list_plots()
for plot in plots:
    print plot.name
    print plot.id
```

This prints the name and ID of plots in the specified project object. An example output is below:

```
GBT Model Visualization Plot
6359881896444898017
RDFR Model Visualization Plot
1355840547833663337
Result - Regression Plot
6712705549326352802
Result - Classification Plot
7303801239242822740
```

`project.get_dataset()`

Use the `get_dataset` function to retrieve a particular dataset using its ID.

Usage

```
dataset_object = project_object.get_dataset(dataset_id)
```

Parameters

- `dataset_object` - Specify the variable that will contain the dataset object
- `project_object` - Specify the project object that contains the dataset you want to retrieve
- `dataset_id` - A string containing the unique identifier for the dataset that you want to retrieve. IDs are shown in the Web interface or can be viewed by printing a dataset or viewing its summary in the SDK.

Example

The following examples retrieves the matrix dataset (whose ID is 6986674145355291507) and stores it to in matrix variable.

```
matrix = ca.get_dataset("6986674145355291507")
```

`project.get_dataset_dependents()`

Use this function to get a list of all downstream (dependent) files related to a dataset. This includes all models, test results, etc, that were created from the dataset.

Usage

```
dataset_object = project_object.get_dataset_dependents(dataset_id)
```

Parameters

- `dataset_object` - Specify the variable that will contain the dataset object
- `project_object` - Specify the project object that contains the dataset you want to retrieve
- `dataset_id` - A string containing the unique identifier for the dataset that you want to retrieve. IDs are shown in the Web interface or can be viewed by printing a dataset or viewing its summary in the SDK.

Example

The example below retrieves all dependent files related to a dataset whose ID is 4274348704094814824.

```
datasetfiles = project.get_dataset_dependents("4274348704094814824")
```

The following example prints the list of files.

```
>>> for dependents in datasetfiles:
...     print dependents
...
Id:4274348704094814824, name=income.data, type=DATASET, is_deletable=True
Id:4426696063906522815, name=income.data.ids, type=DATASET, is_deletable=True
Id:733579628360679490, name=GBT Result, type=TESTRESULT, is_deletable=True
Id:8613266560949776288, name=GBT Model, type=MODEL, is_deletable=True
Id:5350585628603483618, name=GBTR Model, type=MODEL, is_deletable=True
```

`project.get_model()`

Use the `get_model` function to get a model from a project using its ID.

Usage

```
model_object = project_object.get_model(model_id)
```

Parameters

- *model_object* - Specify the variable that will contain the model object
- *project_object* - Specify the project object that contains the model you want to retrieve
- *model_id* - A string containing the unique identifier for the model that you want to retrieve. IDs are shown in the Web interface or can be viewed by printing a model or viewing its summary in the SDK.

Example

The following example gets the `Yearly Income Prediction` model in the `predict` project. The ID for this model is `8534304197005081735`. The model is then stored in the `PredictIncome` variable.

```
PredictIncome = predict.get_model("8534304197005081735")
```

project.get_model_dependents

Use this function to get a list of all downstream (dependent) files related to a model. This includes all test results, plots, etc, that were created from the model.

Usage

```
model_object = project_object.get_model_dependents(model_id)
```

Parameters

- *model_object* - Specify the variable that will contain the model object
- *project_object* - Specify the project object that contains the model you want to retrieve
- *model_id* - A string containing the unique identifier for the model that you want to retrieve. IDs are shown in the Web interface or can be viewed by printing a model or viewing its summary in the SDK.

Example

The example below retrieves all dependent files related to a model whose ID is 8613266560949776288.

```
modelfiles = project.get_model_dependents("8613266560949776288")
```

The following example prints the list of files.

```
>>> for dependents in modelfiles
...     print dependents
...
Id:8613266560949776288, name=GBT Model, type=MODEL, is_deletable=True
Id:733579628360679490, name=GBT Result, type=TESTRESULT, is_deletable=True
```

project.get_result()

Use the `get_result` function to retrieve a result object using its ID.

Usage

```
test_object = project_object.get_result(test_result_id)
```

Parameters

- *test_object* - Specify the variable that will contain the test object
- *project_object* - Specify the project object that contains the result
- *test_result_id* - A string containing the unique identifier for the test result that you want to retrieve. IDs are shown in the Web interface or can be viewed by printing a result or viewing its summary in the SDK.

Example

The following example specifies gets a result file from the `predict` project. The ID for this result is `8443502907664023107`. The result is stored in the `PredictResult` variable.

```
PredictResult = predict.get_result("8534304197005081735")
```

*project.***get_result_dependents()**

Use this function to get a list of all downstream (dependent) files related to a result. This includes all plots and datasets created from this result.

Usage

```
test_object = project_object.get_result_dependents(test_result_id)
```

Parameters

- *test_object* - Specify the variable that will contain the test object
- *project_object* - Specify the project object that contains the result
- *test_result_id* - A string containing the unique identifier for the test result that you want to retrieve. IDs are shown in the Web interface or can be viewed by printing a result or viewing its summary in the SDK.

Example

The example below retrieves all dependent files related to a result whose ID is 733579628360679490.

```
resultfiles = project.get_result_dependents("733579628360679490")
```

The following example prints the list of files.

```
>>> for dependents in resultfiles:
...     print dependents
...
Id:733579628360679490, name=GBT Result, type=TESTRESULT, is_deletable=True
Id:7832448653536604639, name=test_result_source_733579628360679490, type=SOURCE,
is_deletable=True
Id:2224693962685268478, name=test_result_dataset_733579628360679490, type=DATASET,
is_deletable=True
Id:837206533965264181, name=test_result_dataset_733579628360679490 with labels,
type=DATASET, is_deletable=True
Id:8023852568804416979, name=test_probabilities_source_733579628360679490,
type=SOURCE, is_deletable=True
Id:5179394426485077754, name=test_result_dataset_733579628360679490, type=DATASET,
is_deletable=True
Id:3699430448737912851, name=GBT Result plot, type=PLOT, is_deletable=True
```


project.get_plot()

Use the `get_plot` function to retrieve a plot object using its ID.

Usage

```
plot_object = project_object.get_plot(plot_id)
```

Parameters

- *plot_object* - Specify the variable that will contain the plot object
- *project_object* - Specify the project object that contains the result
- *plot_id* - A string containing the unique identifier for the plot that you want to retrieve. IDs are shown in the Web interface or can be viewed by printing a plot or viewing its summary in the SDK.

Example

The following example specifies gets a plot from the `predict` project. The ID for this plot is `1355840547833663337`. The plot is stored in the `plot` variable.

```
plot = predict.get_plot("1355840547833663337")
```

project.get_plot_dependents()

Use this function to get a list of all downstream (dependent) files related to a plot.

Usage

```
plot_object = project_object.get_plot_dependents(plot_id)
```

Parameters

- *plot_object* - Specify the variable that will contain the plot object
- *project_object* - Specify the project object that contains the plot
- *plot_id* - A string containing the unique identifier for the plot that you want to retrieve. IDs are shown in the Web interface or can be viewed by printing a plot or viewing its summary in the SDK.

Example

The example below retrieves all dependent files related to a plot whose ID is 6154555019844760922.

```
plotfiles = project.get_plot_dependents("6154555019844760922")
```

The following example prints the list of files.

```
>>> for dependents in plotfiles:
...     print dependents
..
Id:6154555019844760922, name=GBT Result, type=PLOT, is_deletable=True
```

`project.delete_dataset()`

Use the `delete_dataset` function to delete a dataset along with all downstream files that were generated from this dataset (for example, models and results). We recommend that you review all associated files before deleting a dataset. Refer to [project.get_dataset_dependents\(\)](#).

Note: You cannot delete datasets that were built from test results.

Usage

```
project_id.delete_dataset(dataset_id)
```

Parameters

- `project_object` - Specify the Project Object that contains the dataset you want to delete
- `dataset_id` - Specify the ID that you want to delete. You can obtain the dataset ID using the `list_dataset()` function. (Refer to ["project.list_datasets\(\)" on page 26](#) for more information.)

Example

The following example deletes IncomeData file that was added to a Dataset Object called `dataset`.

```
dataset.delete_dataset("5434731090186968769")
```

`project.delete_model()`

Use the `delete_model` function to delete a model along with all downstream files that were generated from this model (for example, results and plots). We recommend that you review all associated files before deleting a model. Refer to [project.get_model_dependents\(\)](#).

Usage

```
project_id.delete_model(model_id)
```

Parameters

- `project_object` - Specify the Project Object that contains the model you want to delete
- `model_id` - Specify the ID that you want to delete. You can obtain the ID using the `list_models()` function. (Refer to ["project.list_models\(\)" on page 27](#) for more information.)

Example

The following example deletes a model whose ID is 5350585628603483618.

```
project.delete_model("5350585628603483618")
```

project.delete_result()

Use the `delete_result` function to delete a result along with all downstream files that were generated from this result (for example, plots and test result datasets). We recommend that you review all associated files before deleting a result. Refer to [project.get_result_dependents\(\)](#).

Usage

```
project_id.delete_result(result_id)
```

Parameters

- *project_object* - Specify the Project Object that contains the result you want to delete
- *model_id* - Specify the ID that you want to delete. You can obtain the ID using the `list_results()` function. (Refer to ["project.list_results\(\)" on page 28](#) for more information.)

Example

The following example deletes a result whose ID is 733579628360679490.

```
project.delete_result("733579628360679490")
```

`project.delete_plot()`

Use the `delete_plot` function to delete a plot.

Usage

```
project_id.delete_plot(result_id)
```

Parameters

- *project_object* - Specify the Project Object that contains the plot you want to delete
- *model_id* - Specify the ID that you want to delete. You can obtain the ID using the `list_plots()` function. (Refer to "[project.list_models\(\)](#)" on [page 27](#) for more information.)

Example

The following example deletes a plot whose ID is 8434238370819421548.

```
project.delete_plot("8434238370819421548")
```

`project.rename()`

Use this function to change the name and description of an existing project.

Usage

```
project_object.rename(project_name, project_description)
```

Parameters

- *project_object* - The project object to rename
- *project_name* - A string containing the name of the project. This cannot be the name of an other existing project
- *project_description* - A string containing the new description to give the project.

Examples

This example renames the predict project and gives it a description.

```
project.rename("Prediction 2", "Prediction description")
```

`dataset.get_log()`

Use this function to retrieve a log file generated when adding a dataset.

Usage

```
dataset_object.get_log()
```

Parameters

- `dataset_object` - Specify the dataset whose log you want to retrieve

Example

This example below specifies to retrieve the log file generated after the `income.data` dataset was added.

Get the log for the dataset:

```
dataset.get_log()
```

Print the log to your screen:

```
print dataset.get_log().getvalue()
```

Write the log to a file:

```
with open('/Users/<username>/<path>/mylogs.txt','w') as f:  
    f.write(dataset.get_log().getvalue())
```


dataset.**terminate()**

You may find it necessary to cancel a dataset that is currently INPROGRESS. The `terminate` function can be used to stop the dataset creation process. Note that you cannot interrupt datasets that have a status of READY.

Usage

```
dataset_object.terminate()
```

Parameters

- *dataset_object* - Specify the dataset object that you want to interrupt

Example

The following example terminates the creation of the dataset object called `dataset`.

```
dataset.terminate()
```

The response returns True or False, depending on whether the terminate was successful.

`dataset.commit()`

When a dataset is initially added to a project, metadata is computed on a sample of 100,000 rows of the dataset. Use the `commit` function to recompute the metadata across the entire dataset. A dataset is automatically committed prior to creating a machine learning model, so manually calling the `commit` function is often not necessary.

Usage

```
dataset_object.commit().ready()
```

Parameters

- `dataset_object` - The dataset to commit

Note: Only datasets in a READY state can be committed.

Example

The following example commits the income dataset, which is identified with the `dataset` variable.

```
dataset.commit().ready()
```

Run the following to verify that the dataset has been committed.

```
for dataset in ca.list_projects():
    print dataset.name
    print dataset.id
    print dataset.status
```

The output will be similar to the following:

```
income.data
2918893535432269432
{'u'message': u'COMMITTED: Data Set has been validated and is
  ready to use', u'code': u'READY'}
```

dataset.sample()

Use the `sample` function to retrieve a sample set of the data in a READY dataset.

Usage

```
sample_rows = dataset_object.sample(num_rows)
```

Parameters

- *sample_rows* - The returned sample of rows. This is a dictionary with two keys: `headers`, which includes a list of column names of the dataset, and `data`, which contains a list of columns. Each of these is a list containing *num_rows* elements corresponding to the values of that column.
- *dataset_object* - The dataset that you want to sample
- *num_rows* - Specify the number of rows you want to retrieve from the dataset. Lines are automatically taken from the top of the file, so 1 will retrieve the first line, etc., including a header line.

Example

The following example retrieves the first 10 lines from a dataset and stores it in a variable named `sample_lines`.

```
sample_lines = dataset.sample(10)
```

`dataset.summary()`

Use this function with any READY dataset to output the dataset's configuration information to your screen.

Usage

```
print dataset_object.summary()
```

Parameters

- `dataset_object` - The dataset whose configuration you want to review

Example

The example gets a dataset whose ID is 2610786790220353539. This dataset is in the predict project. The dataset object is `dataset`. The configuration information for this model is then printed to the screen.

```
dataset = predict.get_dataset("2610786790220353539")
print dataset.summary()
```

The output will be similar to the following:

```
Skytree dataset summary
-----
id:                2610786790220353539
name:              NewIncomeId
created at:        2015-06-04 16:00:54
updated at:        2015-06-04 16:07:49
source id:         None
state:             Full
type:              Transformed
number of rows:    9769
number of models:  0
-----
status:
message: COMMITTED: Data Set has been validated and is ready to use
code: READY
-----
data columns:

name: age
id: 1
type: Integer
min: 17.0, max: 90.0, mean: 38.5596273928, sigma: 13.6483024067

name: workclass
id: 2
type: Text

name: fnlwt
id: 3
type: Integer
min: 19302.0, max: 1484705.0, mean: 189110.757908, sigma: 106385.277289

name: education
id: 4
type: Text
```

```

name: education-num
id: 5
type: Integer
min: 1.0, max: 16.0, mean: 10.1015457058, sigma: 2.52771830188

name: marital-status
id: 6
type: Text

name: occupation
id: 7
type: Text

name: relationship
id: 8
type: Text

name: race
id: 9
type: Text

name: sex
id: 10
type: Text

name: capital-gain
id: 11
type: Integer
min: 0.0, max: 99999.0, mean: 1028.00511823, sigma: 7096.07469352

name: capital-loss
id: 12
type: Integer
min: 0.0, max: 4356.0, mean: 82.120483161, sigma: 393.628342774

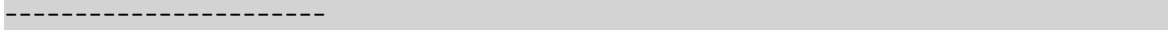
name: hours-per-week
id: 13
type: Integer
min: 1.0, max: 99.0, mean: 40.4288054049, sigma: 12.4447375236

name: native-country
id: 14
type: Text

name: yearly-income
id: 15
type: Text

name: IDs
id: 16
type: Text
min: 0.0, max: 0.0, mean: 0.0, sigma: 0.0
-----
Transform:
-----
Filter: filterExpression lambda IDs: IDs is not None
-----

```



`dataset.add_column()`

Use the `add_column` transform to add a new column to an existing READY dataset. After specifying a column name and the [Lambda expression](#) (see also [Lambda Expressions](#) in The Python Tutorial) to use when calculating values for the new column, the new column will appear as the last column in the dataset.

Usage

```
transform_object = dataset_object.add_column(  
    column_name,  
    lambda_expression,  
    name=new_name  
    config=DatasetConfig(dataset_configuration)  
).ready()
```

Parameters

- *transform_object* - A variable that will contain the transformed dataset
- *dataset_object* - Specify the dataset to transform
- *column_name* - Specify a name for the new column as a string. The column name length cannot exceed 255 characters.
- *lambda_expression* - Specify the method for calculating values in the new column. This is in the form of a string containing a Lambda expression, for example, "lambda age: age + 3". The variable names used in Lambda expressions must match column names in the original dataset.
- *name=new_name* - Optionally specify a name for this transformed dataset. This is passed as a string.
- *dataset_configuration* - Optionally add a `DatasetConfig` object, which allows you to specify an ID column for the dataset as well as a set of columns to consider as free text ('blob' columns). A `DatasetConfig` can be initialized with the following two parameters:
 - *id_column* - a string containing the name of a column to treat as an ID
 - *blob_columns* - a list of strings containing the names of columns to treat as free text

Example

The following example adds a `testColumn` column to the `income.data` dataset. The data in this column takes values using the `lambda age: age + 20` expression.

```
addColumn = income.add_column(  
    "testColumn", "lambda age: age + 20", name="NewColumn"  
).ready()
```

`dataset.add_unique_id_column()`

The `add_unique_id_column` transform adds an ID column as the last column of a dataset. This column serves as a row identifier. The new column is guaranteed to be unique but not necessarily sequential.

Usage

```
transform_object = dataset_object.add_unique_id_column(  
    new_column_name,  
    name=new_name  
    config=DatasetConfig(dataset_configuration)  
).ready()
```

Parameters

- *transform_object* - Specify a variable contain the transformed dataset
- *dataset_object* - Specify the dataset to transform
- *new_column_name* - Specify the name of the new column as a string
- *name=new_name* - Optionally specify a name for this transformed dataset. This is passed as a string.
- *dataset_configuration* - Optionally add a `DatasetConfig` object, which allows you to specify an ID column for the dataset as well as a set of columns to consider as free text ('blob' columns). A `DatasetConfig` can be initialized with the following two parameters:
 - *id_column* - a string containing the name of a column to treat as an ID
 - *blob_columns* - a list of strings containing the names of columns to treat as free text

Example

The following example adds a unique ID column named `IDs` to the dataset in the `income` variable, names the dataset `NewIDColumn`, and stores the transformed dataset in the `uniqueid` variable.

```
uniqueid = income.add_unique_id_column(  
    "IDs", name="NewIDColumn"  
).ready()
```


`dataset.clamp_values()`

The `clamp_values` transform allows you to substitute a new value for values that are out of a specified range. The new value will be either the specified maximum value or the specified minimum value, depending on whether it falls above or below the range.

For example, given a set of data with columns A and B as below:

```
A,B
1,20
2,5
3,0
4,-5
5,-20
```

A clamp transform that specifies minimum and maximum values of column B as -10 and 10, respectively, will transform the data as follows:

```
A,B
1,10
2,5
3,0
4,-5
5,-10
```

Notice that the values that fall outside of the -10, 10 range are clamped with these updated values.

Usage

```
transform_object = dataset_object.clamp_values(
    column_name,
    min,
    max,
    name=new_name
    config=DatasetConfig(dataset_configuration)
).ready()
```

Parameters

- *transform_object* - Specify a variable to contain the transformed dataset
- *dataset_object* - Specify the dataset to transform
- *column_name* - Specify the name of the column that includes the values to be clamped. This is passed as a string.
- *min* - Specify the minimum value to be included in the transform. Values in the column that are smaller than this value will be replaced with this value instead.
- *max* - Specify the maximum value to be included in the transform. Values in the column that are larger than this value will be replaced with this value instead.
- *name=new_name* - Optionally specify a name for this transformed dataset. This is passed as a string.
- *dataset_configuration* - Optionally add a `DatasetConfig` object, which allows you to specify an ID column for the dataset as well as a set of columns to consider as free text ('blob' columns). A `DatasetConfig` can be initialized with the following two parameters:
 - *id_column* - a string containing the name of a column to treat as an ID

- *blob_columns* - a list of strings containing the names of columns to treat as free text

Examples

The following example clamps the value of the "age" column to between 18 and 25.

```
clamp = income.clamp_values(  
    "age", "18", "25", name="Clamped18to25"  
).ready()
```

`dataset.extract_text_features()`

The `extract_text_features` function allows you to transform a dataset by extracting the text features in a specified blob column. These features can be extracted using:

- Term Frequency - Inverse Document Frequency (tf-idf): The IDF value increases proportionally to the number of times a word shows up in the dataset and also diminishes the weight of terms that occur frequently (such as "the" or "and").
- Term Frequency - Bi-Normal Separation (tf-bns): Computes the BNS score for each feature by scaling the magnitude of feature values

Users can also optionally specify the minimum feature weight and the maximum feature size when transforming the dataset.

The resulting dataset will include the featurized words in a new Token column and the weight given to each word in a new Weight column. This new dataset can then be referenced in a `vectorize_text` transform.

Usage

```
transform_object = dataset_object.extract_text_features(  
    blob_column_name,  
    label_column_name,  
    ["tf_idf" | "tf_bns"],  
    minimum_feature_weight=number  
    maximum_feature_size=number  
    name=new_name  
) .ready()
```

Parameters

- *result_object* - Specify a variable to contain the transformed dataset
- *dataset_object* - Specify the dataset to transform. Note that this must be a dataset that has been transformed with ingested text.
- *blob_column_name* - Specify the blob column to be featurized in the new dataset
- *label_column_name* - Specify the label column that was set as categorical text in the `set_as_categorical_text` transform.
- *tf_idf* or *tf_bns* - Specify whether to featurize using inverse document frequency or bi-normal separation.
- *minimum_feature_weight=number* - Optionally specify the minimum weight to be given to a word
- *maximum_feature_size=number* - Optionally specify that maximum size of a word
- *name=new_name* - Optionally specify a name for this transformed dataset. This is passed as a string.

Example

The following example specifies to tokenize the "doc" column in the `categorical` dataset.

```
featurized = categorical.extract_text_features(  
    "doc", "label", "tf_idf", name="Tokenized Text Transform"  
) .ready()
```

`dataset.filter()`

The `filter` transform allows you to filter rows in datasets to include only a subset of the original data in the new, transformed dataset. Filtering rows evaluates a [Lambda expression](#) (see also [Lambda Expressions](#) in The Python Tutorial) on each row of data. The Lambda expression must return a Boolean value. If the returned value is True, then the row is included in the new dataset. If the returned value is False, then the row is filtered out.

For example, given a dataset with a text-type column named "color" the following Lambda expression can be used to filter the data:

This will include only rows where color is red.

```
"lambda color: color == 'red'"
```

This will include all rows except red rows.

```
"lambda color: color != 'red'"
```

Usage

```
transform_object = dataset_object.filter(  
    lambda_expression,  
    name=new_name  
    config=DatasetConfig(dataset_configuration)  
) .ready()
```

Parameters

- *transform_object* - Specify a variable to contain the transformed dataset
- *dataset_object* - Specify the dataset to transform
- *lambda_expression* - Specify the method for calculating values in the new column. This is in the form of a string containing a Lambda expression, for example, "lambda education: education == Masters". The variable names used in Lambda expressions must match column names in the original dataset.
- *name=new_name* - Optionally specify a name for this transformed dataset. This is passed as a string.
- *dataset_configuration* - Optionally add a `DatasetConfig` object, which allows you to specify an ID column for the dataset as well as a set of columns to consider as free text ('blob' columns). A `DatasetConfig` can be initialized with the following two parameters:
 - *id_column* - a string containing the name of a column to treat as an ID
 - *blob_columns* - a list of strings containing the names of columns to treat as free text

Example

The follow examples filters out all rows that include the Private value in the workclass column.

```
filter = income.filter(  
    "lambda workclass: workclass != 'Private'",  
    name="FilteredPrivate"  
) .ready()
```

`dataset.horizontalize()`

When transforming categorical data, the `horizontalize` transform creates a new column for each different value that a categorical column can take. A value of 0.707 is assigned to the right column, and the columns for all other values get 0.

Consider the following example data:

```
blue,1,3
red,4,5
blue,2,7
...
```

When `horizontalize` is applied to the first column, the following transformation take place (assuming that column 1 can only take values of blue and red):

```
0.7071067811865475,0,1,3
0,1,4,5
0.7071067811865475,0,2,7
```

Usage

```
transform_object = dataset_object.horizontalize(
    column_name,
    dictionary=dictionary,
    name=new_name
    config=DatasetConfig(dataset_configuration)
).ready()
```

Parameters

- *transform_object* - Specify a variable to contain the transformed dataset
- *dataset_object* - Specify the dataset to transform
- *column_name* - Specify a name for the new column as a string
- *dictionary=dictionary* - This is optional. When provided, it restricts the new columns to only those values specified by the dictionary. If this option is not specified, then a new column will be created for all unique values that are in the specified column. This is passed as a list of strings.
- *name=new_name* - Optionally specify a name for this transformed dataset. This is passed as a string.
- *dataset_configuration* - Optionally add a `DatasetConfig` object, which allows you to specify an ID column for the dataset as well as a set of columns to consider as free text ('blob' columns). A `DatasetConfig` can be initialized with the following two parameters:
 - *id_column* - a string containing the name of a column to treat as an ID
 - *blob_columns* - a list of strings containing the names of columns to treat as free text

Examples

Example 1

The following example specifies to horizontalize the "workclass" column. The transformation is then applied to the income dataset. As a result, new "workclass.<id>" column will be created.

```
horizontalize = income.horizontalize("workclass", name="Horizontalized").ready()
```

Example 2

Similar to Example 1, this example specifies to horizontalize the "workclass" column. In addition, this example builds a dictionary for the "Federal-gov", "State-gov", and "Local-gov" values, creating new columns for those specific values.

```
dictionary=["Federal-gov", "State-gov", "Local-gov"]  
horizontalize2 = income.horizontalize(  
    "workclass", dictionary, name="Horizontalized"  
).ready()
```

`dataset.ingest_text()`

The `ingest_text` transform defines the blob column (column containing text) in a dataset to be ingested in a new dataset. This transform can only be used with datasets that have one or more blob columns already defined. The resulting dataset will include a new Error column for rows that encounter errors during the transform. If no errors exist, then this transformed dataset will be ready for tokenization. (See [dataset.tokenize_text\(\)](#))

Usage

```
transform_object = dataset_object.ingest_text(  
    blob_column_name, name=new_name, config=DatasetConfig(dataset_  
configuration)  
).ready()
```

Parameters

- *result_object* - Specify a variable to contain the transformed dataset
- *dataset_object* - Specify the dataset to transform
- *blob_column_name* - Specify the blob column to be ingested in the new dataset
- *name=new_name* - Optionally specify a name for this transformed dataset. This is passed as a string.
- *dataset_configuration* - Optionally add a `DatasetConfig` object, which allows you to specify an ID column for the dataset as well as a set of columns to consider as free text ('blob' columns). A `DatasetConfig` can be initialized with the following two parameters:
 - *id_column* - a string containing the name of a column to treat as an ID
 - *blob_columns* - a list of strings containing the names of columns to treat as free text

Example

The following example transforms the `rawReviews` dataset by ingesting the "doc" column. Note that the `rawReviews` already has a blob column defined. (See Examples in [project.create_dataset\(\)](#) (page 23).)

```
ingest_text = raw.ingest_text("doc", name="Ingest Text Transform").ready()
```

`dataset.ingest_url()`

For datasets that contain a list of URLs, `ingest_url` transforms the dataset by ingesting the text from the URL links in the new dataset. This transform can only be used with datasets that have one or more blob columns already defined. The resulting dataset will include a new Error column for rows that encounter errors during the transform. If no errors exist, then this transformed dataset will be ready for tokenization. (See [dataset.tokenize_text](#).)

Usage

```
transform_object = dataset_object.ingest_url(  
    blob_column_name, name=new_name, config=DatasetConfig(dataset_  
configuration)  
) .ready()
```

Parameters

- *result_object* - Specify a variable to contain the transformed dataset
- *dataset_object* - Specify the dataset to transform
- *blob_column_name* - Specify the blob column to be ingested in the new dataset
- *name=new_name* - Optionally specify a name for this transformed dataset. This is passed as a string.
- *dataset_configuration* - Optionally add a `DatasetConfig` object, which allows you to specify an ID column for the dataset as well as a set of columns to consider as free text ('blob' columns). A `DatasetConfig` can be initialized with the following two parameters:
 - *id_column* - a string containing the name of a column to treat as an ID
 - *blob_columns* - a list of strings containing the names of columns to treat as free text

Example

The following example transforms the `urlRaw` dataset by ingesting the "doc" blob column that was specified when the dataset was initially added. (See Examples in [project.create_dataset\(\)](#).)

```
ingest_url = urlRaw.ingest_url(  
    "doc", name="URL Ingest Transform"  
) .ready()
```


`dataset.inner_join()`

The `inner_join` transform combines like entries from two datasets into a single dataset. The dataset object for both datasets is required.

Note: Joins are only supported on two datasets in a single transform. Create multiple transforms to join more than two datasets. In addition, duplicate column names cannot exist in either dataset. Finally, join transforms cannot be included in multi-step transforms.

Usage

```
transform_object = dataset_object1.inner_join(  
    dataset_object2,  
    [column_name1,  
    column_name2],  
    name=new_name  
    config=DatasetConfig(dataset_configuration)  
).ready()
```

Parameters

- *transform_object* - Specify a variable to contain the transformed dataset
- *dataset_object1* - Specify the first dataset to join
- *dataset_object2* - Specify the second dataset to join
- *column_name1* - Specify the column name from the first dataset that you want to be joined
- *column_name2* - Specify the column name from the second dataset that you want to be joined
- *name=new_name* - Optionally specify a name for this transformed dataset. This is passed as a string.
- *dataset_configuration* - Optionally add a `DatasetConfig` object, which allows you to specify an ID column for the dataset as well as a set of columns to consider as free text ('blob' columns). A `DatasetConfig` can be initialized with the following two parameters:
 - *id_column* - a string containing the name of a column to treat as an ID
 - *blob_columns* - a list of strings containing the names of columns to treat as free text

Example

The following performs an `inner_join` transform on the `join1` dataset. This dataset includes a `color` column with the following values: red, blue, red, yellow, orange, and pink. The example specifies to join the `color1` column from the `join2` dataset. The `color1` column has the following values: red, blue, red, and purple. The resulting dataset, `InnerJoin`, will combine the two datasets and include only those entries that have common values. In other words, only red and blue entries will appear in the new dataset.

```
ij = join1.inner_join(  
    join2, ['color', 'color1'], name="InnerJoin"  
).ready()
```

`dataset.left_outer_join()`

Unlike the `inner_join()` transform, which combines like values from two datasets, a `left_outer_join` will add values from Dataset2 into Dataset1. The resulting dataset returns all values from an inner join plus values in Dataset1 that do not match Dataset2, including rows with NULL values.

Note: Joins are only supported on two datasets in a single transform. Create multiple transforms to join more than two datasets. In addition, duplicate column names cannot exist in either dataset. Finally, join transforms cannot be included in multi-step transforms.

Usage

```
transform_object = dataset_object1.left_outer_join(  
    dataset_object2,  
    [column_name1,  
    column_name2],  
    name=new_name  
    config=DatasetConfig(dataset_configuration)  
).ready()
```

Parameters

- *transform_object* - Specify a variable to contain the transformed dataset
- *dataset_object1* - Specify the first dataset to join
- *dataset_object2* - Specify the second dataset to join
- *column_name1* - Specify the column name from the first dataset that you want to be joined
- *column_name2* - Specify the column name from the second dataset that you want to be joined
- *name=new_name* - Optionally specify a name for this transformed dataset. This is passed as a string.
- *dataset_configuration* - Optionally add a `DatasetConfig` object, which allows you to specify an ID column for the dataset as well as a set of columns to consider as free text ('blob' columns). A `DatasetConfig` can be initialized with the following two parameters:
 - *id_column* - a string containing the name of a column to treat as an ID
 - *blob_columns* - a list of strings containing the names of columns to treat as free text

Example

The following performs a `left_outer_join` transform on the `join1` and `join2` datasets based on the `color` column. The resulting dataset, `LeftOuterJoin`, will combine the `color` columns into a single column. This will be the first column in the table. The remaining columns will include the columns from `join1` (which include all data) followed by columns from `join2` that match the `color` criteria (and may contain null values).

```
loj = join1.left_outer_join(  
    join2, ['color', 'color1'], name="LeftOuterJoin"  
).ready()
```

`dataset.normalize()`, `dataset.normalize_unit()`, `dataset.normalize_standard()`

The normalize functions are only applicable to columns containing numeric data. These allow you to normalize columns containing numeric data. Two types of normalization are available:

- **Unit:** This form makes the data range between 0 and 1. Min and max values can be provided when performing a Unit normalization. When these are provided, the column is recalculated using $[(value - min_value) / max_value]$.
- **Standard:** This gives the data a mean value of 0 and unit variance. Standard deviation and mean values can be provided when performing Standard normalization. When these are provided, the column is recalculated using $[(value - stdev_value) / mean_value]$.

Usage

Three functions are available for normalizing columns:

`dataset.normalize()`

Specify the type within the transform. When this method is performed, min/max and stdev/mean values are not provided.

```
transform_object = dataset_object.normalize(  
    normalization_type,  
    normalization_column,  
    name=new_name  
    config=DatasetConfig(dataset_configuration)  
) .ready()
```

`dataset.normalize_unit()`

Specify min/max values for Unit normalization.

```
transform_object = dataset_object.normalize_unit(  
    normalization_column,  
    min,  
    max,  
    name=new_name  
    config=DatasetConfig(dataset_configuration)  
) .ready()
```

`dataset.normalize_standard()`

Specify stdev/mean values for Standard normalization.

```
transform_object = dataset_object.normalize_standard(  
    normalization_column,  
    stdev,  
    mean,  
    name=new_name  
    config=DatasetConfig(dataset_configuration)  
) .ready()
```

Parameters

- `transform_object` - Specify a variable to contain the transformed dataset
- `dataset_object` - Specify the dataset to transform

- *normalization_type* - Specify Unit or Standard
 - *min* - If Unit is specified as the normalization type, then optionally specify a number for the minimum value that will be in this column. If this is specified, then *max* must also be specified.
 - *max* - If Unit is specified as the normalization type, then optionally specify a number for the maximum value that will be in this column. If this is specified, then *min* must also be specified.
 - *stdev* - If Standard is specified as the normalization type, then optionally specify a number for the standard deviation value to be used in the transform. If this is specified, then *mean* must also be specified.
 - *mean* - If Standard is specified as the normalization type, then optionally specify a number for the mean value to be used in the transform. If this is specified, then *stdev* must also be specified.
- *column_name* - Specify the column name that you want to normalize
- *name=new_name* - Optionally specify a name for this transformed dataset. This is passed as a string.
- *dataset_configuration* - Optionally add a DatasetConfig object, which allows you to specify an ID column for the dataset as well as a set of columns to consider as free text ('blob' columns). A DatasetConfig can be initialized with the following two parameters:
 - *id_column* - a string containing the name of a column to treat as an ID
 - *blob_columns* - a list of strings containing the names of columns to treat as free text

Examples

Example 1

The following example specifies to normalize the "hours-per-week" column using Standard normalization without any additional calculations performed on the values.

```
normalize1 = income.normalize(
    "Standard", "hours-per-week", name="NormStd"
).ready()
```

Example 2

This example specifies to normalize the "hours-per-week" column and recalculate the data based on specified min and max values. The minimum and maximum values to be included are 0 and 20.

```
normalize2 = income.normalize_unit(
    "hours-per-week", "0", "20", name="NormUnit"
).ready()
```

`dataset.remove_columns()`

Use the `remove_columns` transform to exclude certain columns based on the column name. Only datasets that have header rows (column names) can be referenced in this transform.

Usage

```
transform_object = dataset_object.remove_columns(  
    [column_names],  
    name=new_name  
    config=DatasetConfig(dataset_configuration)  
).ready()
```

Parameters

- *transform_object* - Specify a variable to contain the transformed dataset
- *dataset_object* - Specify the dataset to transform
- *column_names* - Specify the columns to remove, as a list of strings.
- *name=new_name* - Optionally specify a name for this transformed dataset. This is passed as a string.
- *dataset_configuration* - Optionally add a `DatasetConfig` object, which allows you to specify an ID column for the dataset as well as a set of columns to consider as free text ('blob' columns). A `DatasetConfig` can be initialized with the following two parameters:
 - *id_column* - a string containing the name of a column to treat as an ID
 - *blob_columns* - a list of strings containing the names of columns to treat as free text

Example

This example excludes the "marital-status" and "age" columns from the `IncomeData` dataset.

```
removecolumns = income.remove_columns(  
    ["yearly-income", "marital-status"], name="RemovedColumns"  
).ready()
```

`dataset.right_outer_join()`

A `right_outer_join` transform behaves similarly to the `left_outer_join()` transform, except that the treatment of the table is reversed. Every row from Dataset2 will appear in the joined table at least once. If a matching row from Dataset1 does not exist, then those values will be empty (or NULL).

Note: Joins are only supported on two datasets in a single transform. Create multiple transforms to join more than two datasets. In addition, duplicate column names cannot exist in either dataset. Finally, join transforms cannot be included in multi-step transforms.

Usage

```
transform_object = dataset_object1.right_outer_join(  
    dataset_object2,  
    [column_name1, column_name2],  
    name=new_name  
    config=DatasetConfig(dataset_configuration)  
).ready()
```

Parameters

- `transform_object` - Specify a variable to contain the transformed dataset
- `dataset_object1` - Specify the first dataset to join
- `dataset_object2` - Specify the second dataset to join
- `column_name1` - Specify the column name from the first dataset that you want to be joined
- `column_name2` - Specify the column name from the second dataset that you want to be joined
- `name=new_name` - Optionally specify a name for this transformed dataset. This is passed as a string.
- `dataset_configuration` - Optionally add a `DatasetConfig` object, which allows you to specify an ID column for the dataset as well as a set of columns to consider as free text ('blob' columns). A `DatasetConfig` can be initialized with the following two parameters:
 - `id_column` - a string containing the name of a column to treat as an ID
 - `blob_columns` - a list of strings containing the names of columns to treat as free text

Example

The following performs a `right_outer_join` transform on the `join1` and `join2` datasets based on the `color` column. The resulting dataset, `RightOuterJoin`, will combine the `color` columns into a single column. This will be the first column in the table. The remaining columns will include the columns from `join1` that match the `color` criteria (including null values) followed by columns from `join2` (which will include all data).

```
roj = join1.right_outer_join(  
    join2, ['color', 'color1'], name="RightOuterJoin"  
).ready()
```

`dataset.set_as_categorical_number()`

The `set_as_categorical_number` transform allows you to treat each entry in a column as a numeric category.

Consider example data as follows:

```
0
00
10
```

By default, the data will be treated as a continuous value.

```
0.0
0.0
10.0
```

With numeric categories, 0 and 00 both get treated as 0, the same category.

```
0
0
1
```

Usage

```
transform_object = dataset_object.set_as_categorical_number(
    column_name, name=new_name, config=DatasetConfig(dataset_configuration)
).ready()
```

Parameters

- *transform_object* - Specify a variable to contain the transformed dataset
- *dataset_object* - Specify the dataset to transform
- *column_name* - Specify a name for the new column as a string
- *name=new_name* - Optionally specify a name for this transformed dataset. This is passed as a string.
- *dataset_configuration* - Optionally add a `DatasetConfig` object, which allows you to specify an ID column for the dataset as well as a set of columns to consider as free text ('blob' columns). A `DatasetConfig` can be initialized with the following two parameters:
 - *id_column* - a string containing the name of a column to treat as an ID
 - *blob_columns* - a list of strings containing the names of columns to treat as free text

Example

The following example transforms the `income` dataset by specifying that the `education-num` column should be treated as a categorical number.

```
catnum = income.set_as_categorical_number(
    "education-num", name="CategoricalNumber"
).ready()
```

`dataset.set_as_categorical_text()`

The `set_as_categorical_text` transform allows you to specify that data that appears numerical should instead be treated as text (for example, as a product code).

Consider example data as follows:

```
0
00
10
```

By default, the data will be treated as a continuous value.

```
0.0
0.0
10.0
```

Setting the column as categorical text will treat it as categorical. Note that 00 is not the same text as 0.

```
"0"
"00"
"10"
```

Note: For columns with more than 2048 categorical values, the column is converted to non-categorical text in order to limit the dictionary size.

Usage

```
transform_object = dataset_object.set_as_categorical_text(
    column_name, name=new_name, config=DatasetConfig(dataset_configuration)
).ready()
```

Parameters

- *result_object* - Specify a variable to contain the transformed dataset
- *dataset_object* - Specify the dataset to transform
- *column_name* - Specify a name for the column as a string
- *name=new_name* - Optionally specify a name for this transformed dataset. This is passed as a string.
- *dataset_configuration* - Optionally add a `DatasetConfig` object, which allows you to specify an ID column for the dataset as well as a set of columns to consider as free text ('blob' columns). A `DatasetConfig` can be initialized with the following two parameters:
 - *id_column* - a string containing the name of a column to treat as an ID
 - *blob_columns* - a list of strings containing the names of columns to treat as free text

Examples

Example 1

The following example transforms the `income` dataset by specifying that the `education-num` column should be treated as categorical text.


```
cattext = income.set_as_categorical_text(  
    "education-num", name="CategoricalText"  
).ready()
```

Example 2

The following example specifies to set a column named "label" as categorical text. After this completes successfully, the text dataset can then be featurized.

```
categorical = tokenized.set_as_categorical_text(  
    "label", name="Set Label as Categorical Text"  
).ready()
```

`dataset.set_as_id()`

The `set_as_id` transform sets an existing column of a dataset as the ID column. The ID column should contain a unique identifier for each row, though that is not checked. It is used to match up the predicted values on a dataset with the original rows. If an ID column was not specified via a `DatasetConfig` when the dataset was created, it can be assigned using the `set_as_id` transform.

Usage

```
transform_object = dataset_object.set_as_id(  
    column_name, name=new_name  
) .ready()
```

Parameters

- `transform_object` - Specify a variable to contain the transformed dataset
- `dataset_object` - Specify the dataset to transform
- `column_name` - Specify the column that will be the ID column. This is passed as a string.
- `name=new_name` - Optionally specify a name for this transformed dataset. This is passed as a string.

Example

The following example transforms the `income` dataset by specifying the `"ids"` column as the ID column. The new dataset is `"dataset with IDs"`.

```
yearlyincomeid = income.set_as_id("ids", name="dataset with IDs").ready()
```

`dataset.split()`

The `split` transform allows you to split an existing dataset into multiple datasets.

Note: Split transforms cannot be included in multi-step transforms.

Usage

When performing a split transform, you can specify to unpack multiple datasets or to return a list.

Return multiple datasets:

```
split_transform_tuple = dataset_object.split(  
    split_ratios,  
    split_seed,  
    names=[new_names]  
)
```

Return a list:

```
split_transform_list = dataset_object.split(  
    split_ratios,  
    split_seed,  
    names=[new_names]  
)
```

Parameters

- `split_transform_tuple` - A variable containing the transformed tuple of split datasets. This must contain a number of datasets equal to the number specified by the `split_ratios` list. It is recommended to immediately unpack the result, e.g.
`t1, t2 = dataset_object.split(split_ratios, split_seed)`. This can be used instead of `split_transform_list`.
- `split_transform_list` - A variable containing a list of the split datasets. This can be used instead of `split_transform_tuple`. This option allows users to collect the split datasets without unpacking.
- `dataset_object` - Specify the dataset to split
- `split_ratios` - Specify the ratio of each split. For example, if you want to specify a file be split into two equal parts, then this value would be `[1, 1]`. Similarly, if you want a file to be split in half, with the second half split in two equal parts, then this value would be `[2, 1, 1]`. Floating values are also accepted. For example, to perform an 80%/20% split, you can specify `[0.8, 0.2]`.
- `split_seed` - Optionally specify a seed value for the random shuffle. A random seed will be used if this is not provided.
- `names=[new_names]` - Optionally specify a list of names for each split in the `transform_tuple`. These names are passed as strings.

Example

The following example splits a dataset into three equal parts, resulting in three transformed datasets.

```
split1, split2, split3 = income.split(  
    [1.0, 1.0, 1.0],  
    names=["split1", "split2", "split3"]  
)
```

```
split1 = split1.ready()  
split2 = split2.ready()  
split3 = split3.ready()
```

`dataset.tokenize_text`

This transformation tokenizes the blob column in a text dataset. Note that this can only be used on datasets that have been transformed already with ingested text. After this completes successfully, you can set this column as categorical and then extract the text features.

Usage

```
transform_object = dataset_object.tokenize_text(  
    blob_column_name, name=new_name, config=DatasetConfig(dataset_  
configuration)  
).ready()
```

Parameters

- *result_object* - Specify a variable to contain the transformed dataset
- *dataset_object* - Specify the dataset to transform. Note that this must be a dataset that has been transformed with ingested text.
- *blob_column_name* - Specify the blob column to be tokenized in the new dataset
- *name=new_name* - Optionally specify a name for this transformed dataset. This is passed as a string.
- *dataset_configuration* - Optionally add a `DatasetConfig` object, which allows you to specify an ID column for the dataset as well as a set of columns to consider as free text ('blob' columns). A `DatasetConfig` can be initialized with the following two parameters:
 - *id_column* - a string containing the name of a column to treat as an ID
 - *blob_columns* - a list of strings containing the names of columns to treat as free text

Example

The following example specifies to tokenize the "doc" column in the ingest dataset.

```
tokenized = ingest.tokenize_text(  
    "doc", name="Tokenized Text Transform"  
).ready()
```

`dataset.vectorize_text()`

This transform vectorizes the features in a dataset. This transform is done on a dataset that includes a label column. A dataset that was transformed using `extract_text_features` is also required.

Usage

```
transform_object = dataset_object.vectorize_text(  
    blob_column_name,  
    label_column_name,  
    extracted_features_dataset_object  
    maximum_feature_size=number  
    name=new_name,  
    config=DatasetConfig(dataset_configuration)  
).ready()
```

Parameters

- *result_object* - Specify a variable to contain the transformed dataset
- *dataset_object* - Specify the dataset to transform. Note that this must be a dataset that has been transformed with a label column set as categorical text.
- *blob_column_name* - Specify the blob column to be vectorized in the new dataset
- *label_column_name* - Specify the label column to be vectorized in the new dataset
- *extracted_features_dataset_object* - Specify the dataset that was returned by the `extracted_features` transform. (See [dataset.extract_text_features\(\)](#))
- *name=new_name* - Optionally specify a name for this transformed dataset. This is passed as a string.
- *dataset_configuration* - Optionally add a `DatasetConfig` object, which allows you to specify an ID column for the dataset as well as a set of columns to consider as free text ('blob' columns). A `DatasetConfig` can be initialized with the following two parameters:
 - *id_column* - a string containing the name of a column to treat as an ID
 - *blob_columns* - a list of strings containing the names of columns to treat as free text

Example

The following example creates a vectorized dataset using the labels from the categorical dataset and the features from the featurized dataset.

```
vectorized = categorical.vectorize_text(  
    "doc", "label", featurized, name="Vectorized Text Transform"  
).ready()
```

`dataset.apply_transform_steps()`

The `apply_transform_steps` function allows you to apply multiple transform types to a dataset. For example, you might to filter a dataset based on an age while ignoring the top 100 rows in a dataset. The transformations will be applied in the order that they are specified in the configuration file.

Note: The join transforms, the split transform, and the set as ID transform cannot be used when applying multiple transforms to a dataset.

Usage

Create the transform configuration:

```
transform_configuration_object = TransformSteps()
transform_configuration_object.add_transform_function(value)
```

Apply the configuration

```
transformed_dataset = dataset_object.apply_transform_steps(
    transform_configuration_object, name
).ready()
```

Parameters

Configuration:

- *transform_configuration_object* - Specify a variable that will serve as the configuration object
- *transform_function* - Specify the transform to apply. Accepted values include the following:

```
add_unique_id
add_clamp_values
add_filter
add_horizontalize
add_new_column
add_normalize
add_normalize_unit
add_normalize_standard
remove_columns
add_categorical_number
add_categorical_text
add_ingest_text
add_ingest_url
tokenize_text
extract_text_features
vectorize_text
```

- *value* - Specify the value for the transform. Values vary based on the type of *transform_function* that is specified. For example, `remove_columns` accepts a column name, `add_clamp_values` requires a column name as well as min and max values, etc.

Transform the dataset:

- *transformed_dataset* - Specify a variable that serve as the transformed dataset object
- *dataset_object* - Specify the dataset to transform
- *transformed_configuration_object* - Specify the configuration that includes the transforms to be used when creating the new dataset.
- *new_name* - Optionally specify a name for this transformed dataset. This is passed as a string.

Example

The following example applies multiple transforms to a dataset using a single configuration file. The configuration specifies to horizontalize the `workclass` column, then apply a filter to the `age` column, and finally to remove the `fnlwt` column.

Create the configuration:

```
tr = TransformSteps()
tr.add_horizontalize("workclass")
tr.add_filter("lambda age: age > 50")
tr.remove_columns(["fnlwt"])
```

Transform the dataset:

```
transformed = income.apply_transform_steps(tr, "multi-transformed").ready()
```


Dataset.apply_custom_transform()

The `Dataset.apply_custom_transform` function allows you to reference a Spark script written in Python within a transform. This allows you to access many additional Spark functions not directly encompassed in Skytree transforms.

Applying a custom transform is done in three steps:

1. Create a script file in Python.
2. Upload the script file in a custom transform using [skytree.add_custom_transform\(\)](#)
3. Apply the script to a dataset.

Note: Refer to [Custom Script Objects](#) (page 149) for a list of objects and variables available for custom scripts.

Usage

```
custom_transform_object = Dataset.apply_custom_transform(  
    custom_script_name, [dataset_objects], [new_name], [args]  
)
```

Parameters

- *custom_transform_object* - Specify a variable to contain the transformed dataset
- *custom_script_name* - A string containing the name of the script that this custom transform will use
- *dataset_objects* - A list including the dataset object or objects that will be used when transforming the final dataset.
- *new_name* - Optionally specify a name for this transformed dataset. This is passed as a string.
- *args* - Provide the required arguments for the script as a list of strings. This list varies based on the script included in the *custom_script_name*.

Examples

The example below transforms the income dataset using the `find_replace.py` script. This script was added previously using [skytree.add_custom_transform\(\)](#). The arguments that are provided tell the script to replace all instances of United-States in column 14 with USA. (Note that column index values start at 0.)

```
custom_dataset = Dataset.apply_custom_transform(  
    custom_transform,  
    [income],  
    ["find-replace-dataset"],  
    ["United-States", "USA", 13]  
)
```

Note: The `Dataset.apply_custom_transform()` function returns a list of a dataset. The `ready()` function cannot be used with this.

Chapter 4 The skytree.prediction Module

The **skytree.prediction** module includes a function for creating models.

- `skytree.prediction.learn()`

Model Instance Methods

The following instance methods are used when working with models. In addition, after models are created, a variety of attributes become available. Refer to [Model Properties](#) (page 78) for more information.

- `model.terminate()`
- `model.summary()`
- `model.get_log()`
- `model.get_pmml_gzip()`
- `model.get_pdp_variable_names()`
- `model.visualize_pdp()`
- `model.test()`

Test Results Instance Methods

The following instance methods are used when working with tests. After test results are added to a project, a variety of attributes become available. Refer to [Test Result Properties](#) (page 79) for more information. In addition, if a classification score configuration is used, an additional set of attributes becomes available. Refer to [Classification Score Configuration Properties](#) (page 81) for more information.

- `test.create_probabilities_dataset()`
- `test.create_labels_dataset()`
- `test.create_targets_dataset()`
- `test.terminate()`
- `test.summary()`
- `test.visualize()`
- `test.get_log()`

- `test.get_probabilities()`
- `test.get_labels()`
- `test.get_targets()`

Plot Registry Entity Instance Methods

The following instance methods are used when working with plots.

- `PlotRegistry()`
- `plot_registry_entity.findall()`
- `plot_registry_entity.findById()`
- `plots.terminate()`

Finally, a `ready()` function should be included with every input request. The `ready()` function is a blocking call that polls the server continuously until the resource is available for use (i.e, has a READY status). Models, tests, and datasets must be READY before they can be used.

Model Properties

After a model is built, the following properties can be used to retrieve specific information.

Table 4: Model Properties

Property	Description
<code>model.id</code>	Shows the model ID.
<code>model.name</code>	Shows the model name.
<code>model.dataset_id</code>	Shows ID of the dataset that was used when training the model.
<code>model.status</code>	Shows the model's status (for example, READY).
<code>model.method</code>	Shows the method that was used to build the model.
<code>model.tuning_results</code> <code>[n].parameters</code>	For models that were trained using tuning parameters, this shows parameters for the individual iterations in the tuning results. <i>n</i> indicates the specific point that you want to view.
<code>model.tuning_results</code> <code>[n].scores</code>	For models that were trained using tuning parameters, this shows scoring information for the individual iterations in the tuning results. <i>n</i> indicates the specific point that you want to view.
<code>model.train_config</code> <code>print model.train_config</code>	Shows the training parameters that were used to build this model.
<code>model.model_config</code> <code>print model.model_</code> <code>config</code>	Use this attribute to create a configuration instance for a specified model. Once created, you can use <code>print model.model_config</code> to view the configuration that was used when building this model. The output corresponds to the <i>configuration_object</i> from the <code>skytree.prediction.learn()</code> function.
<code>model.rest_api_</code> <code>response_dict</code>	Shows the dictionary entries for each configuration option in the model.

Table 4: Model Properties (continued)

Property	Description
<code>model.created_at</code>	Shows when the specified model was created, including the month, date, year, hour, minutes, and seconds.
<code>model.updated_at</code>	Shows when the specified model was last updated, including the month, date, year, hour, minutes, and seconds.
<code>model.summary</code> <code>print model.summary()</code>	Use this attribute to create a configuration instance for a specified model. Once created, you can use <code>print model.summary()</code> to view the model configuration information.

Test Result Properties

After a test result is generated, the following properties can be used to retrieve specific information.

Table 5: Test Result Properties - General. These properties can be used with classification and regression test results.

Property	Description
<code>test_result.id</code>	Shows the test ID.
<code>test_result.name</code>	Shows the test result name.
<code>test_result.status</code>	Shows the test results's status (for example, READY).
<code>test_result.dataset_id</code>	Shows ID of the dataset that was used to compare against a model.
<code>test_result.model_id</code>	Shows the ID of the model that was used for this test result.
<code>test_result.project_id</code>	Shows the ID of the project that contains this test result.
<code>test_result.rest_api_response_dict</code>	Shows the dictionary entries scores in the test result.
<code>test_result.created_at</code>	Shows when the specified test result was created, including the month, date, year, hour, minutes, and seconds.
<code>test_result.updated_at</code>	Shows when the specified test result was last updated, including the month, date, year, hour, minutes, and seconds.

Table 6: Test Result Properties - Classification Test Result. These properties can only be used with classification test results

Property	Description
<code>classification_result.false_positive_rate</code>	Shows the false positive rate. This represents the number of FP (points mistakenly labeled 1) divided by the number of points not belonging to class 1

Table 6: Test Result Properties - Classification Test Result. These properties can only be used with classification test results (continued)

Property	Description
<code>classification_result.confusion_matrix</code> <code>print classification_result.confusion_matrix</code>	Shows the result's confusion matrix. Each column of the matrix represents the instances in a predicted class, while each row represents the instances in an actual class. For binary classifiers, the confusion matrix summarizes the number of correct and false classifications in both directions (i.e., both true and false positives and negatives).
<code>classification_result.precision</code>	Shows the test result's precision value. This is the number of TPs (true positives) divided by the number of points classified as belonging to class 1 (TP+FP).
<code>classification_result.fscore</code>	Shows the test result's fscore. This is $(2 \times \text{precision_recall}) / (\text{precision} + \text{recall})$.
<code>classification_result.recall</code>	Shows the test result's recall value. This shows the true positive rate: TP (points correctly labeled 1) divided by the actual number of points of class 1.
<code>classification_result.accuracy</code>	Shows the test result's accuracy value. This is the number of correctly classified points for both classes (TP+TN) divided by the total number of points.
<code>classification_result.gini</code>	Shows the test result's Gini value.
<code>classification_result.capture_deviation</code>	Shows the test result's capture deviation value.
<code>classification_result.score_config</code> <code>print classification_result.score_config</code>	Prints the scoring configuration options (if any) that were provided when this test result was created.

Table 7: Test Result Properties - Regression Test Result. These properties can only be used with regression test results

Property	Description
<code>regression_result.l1_error</code>	Shows the test result's L1 error.
<code>regression_result.relative_l1_error</code>	Shows the test result's relative L1 error.
<code>regression_result.l2_error</code>	Shows the test result's L2 error.
<code>regression_result.relative_l2_error</code>	Shows the test result's relative L2 error.
<code>regression_result.mean_squared_error</code>	Shows the test result's mean squared error.
<code>regression_result.root_mean_squared_error</code>	Shows the test result's root mean squared error.
<code>regression_result.coefficient_of_determination</code>	Shows the test result's coefficient of determination
<code>regression_result.mean_absolute_error</code>	Shows the test result's mean absolute error

Classification Score Configuration Properties

When a classification score configuration object is used during a test result, the following properties can be used to retrieve specific information.

Table 8: *Model Properties*

Property	Description
<code>result.quantiles</code>	Shows the quantiles int value used in the result.
<code>result.class_weights</code>	Shows the list of class weights, one per label class, in the labels output
<code>result.k_for_precision</code>	Shows the k precision double value used in the result.

skytree.prediction.learn()

The `skytree.prediction.learn` function is used to build models for any machine learning method. Each Skytree machine learning method utilizes a unique configuration class: `RdfConfig`, `GbtConfig`, `RdfrConfig`, `GbtrConfig`, `SvmConfig`, and `AutoModelConfig`. These methods are described in [Methods](#) (page 113). The configuration classes control the behavior of the `skytree.prediction.learn` function, allowing the user to specify the model to learn and its properties. The options that can be included in each configuration are described in [Model Configuration](#) (page 117).

Note: An ID is required when training and testing models. We recommend that you perform the `set_as_id` transform as the final transform before training or testing a model. See [dataset.set_as_id\(\)](#) (page 68).

Usage

Create the configuration:

```
configuration_object = method.Config()
configuration_object.param = value
```

Train the model

```
model_object = skytree.prediction.learn(
    dataset_object,
    objective_column,
    configuration_object,
    name=name
).ready()
```

Parameters

Configuration:

- `configuration_object` - Specify a variable that will serve as the configuration object
- `method` - Specify the method to use when building the model. Options include: `gbt`, `gbtr`, `rdf`, `rdfr`, `svm`, and `autoModel` (for example, `rdf.Config`)
- `param=value` - Specify the configuration parameters and values to use during model training. Parameters for different types of models and their allowable values are described in the [Model Configuration](#) section.

Build the model:

- `model_object` - Specify a variable that serve as the model object
- `dataset_object` - Specify the dataset that contains the labeled training data for the model
- `objective_column` - A string specifying the column in the dataset that will be the target column. This is the column that you are attempting to predict. When performing a classification problem, this column must be a categorical column with two unique values. When performing a regression problem, this column must include only numeric values. Finally, the `objective_column` cannot include missing values. An error will result if the specified `objective_column` does not meet this criteria.
- `configuration_object` - Specify the configuration that includes the parameter values to be used when training the model. Refer to the [Model Configuration](#) (page 117) for the parameters that can be specified for each machine learning method.
- `name=name` - A string containing the name to assign to this model (optional).

Example

The following example trains a model in the predict project. This model is based on the `income.data` dataset, whose ID is 8483059517210011871. A configuration is created using GBT as the machine learning method. This configuration specifies to build 4 trees (`num_trees = [4]`). The "yearly-income" column in the dataset is specified as the objective column.

Note: Refer to [Model Examples](#) (page 142) for additional examples.

Get the Predict project:

```
predict = skytree.get_project("2628892886694058262")
```

Get the income.data dataset:

```
incomeid = predict.get_dataset("8483059517210011871")
```

Create the configuration:

```
configuration = gbt.Config()  
configuration.num_trees = [4]
```

Train the model:

```
GbtModel = skytree.prediction.learn(  
    incomeid, "yearly-income", configuration, name="GbtModel"  
)  
.ready()
```

`model.terminate()`

You may find it necessary to cancel a model that is currently INPROGRESS. The `terminate` function can be used to stop the creation process. Note that you cannot interrupt models that have a status of READY.

Usage

```
model_object.terminate()
```

Parameters

- `model_object` - Specify the model to interrupt

Example

The following example terminates a `GbtrModel` model that is being created.

```
GbtrModel.terminate()
```

The response returns `True` or `False`, depending on whether the `terminate` was successful.

`model.summary()`

Use `summary` with any READY model to output the model configuration information to your screen. If tuning was performed, then this will also output the tuning results.

Usage

```
print model_object.summary()
```

Parameters

- `model_object` - The model that you want to retrieve

Example

The example gets a model whose ID is 7669976834524949568. This model is in the `predict` project. The model object is `gbtmodel`. The results for this model are then printed to the screen.

```
gbtmodel = predict.get_model("7669976834524949568")
print gbtmodel.summary()
```

The output will be similar to the following:

```
Skytree gbt model summary
Smart search seed:          None
-----
id:          7669976834524949568
name:        income test
method:      gbt
created at:  2015-05-29 12:40:21
updated at:  2015-05-29 12:57:43
dataset id:  8726297981033615470
-----
status:
message: Model is ready to be used
code: READY
-----
tuning results:
Skytree gbt configuration summary
-----
Objective Column:          yearly-income
Number of folds:           None
Holdout ratio:             None
Output PMML:               False
Sampling with replacement:  None
Num trees:                  [100]
Tree depth:                 [3]
Num dimensions:            Default
Holdout seed:               None
Categorical selection method: None
Categorical random split tries: None
Categorical random split threshold: None
Dimension sampling seed:    None
Table sampling seed:        None
Categorical sampling seed:  None
Testing objective:          None
Compression:                None
Max splits:                 None
```

```
Min node size:                None
Cardinality Based Dim Sampling:  None
Smart search:                  False
Smart search iterations:        None
-----
Imbalance:                     False
Imbalance scale:               None
Ensemble size:                 None
Learning rate:                  [0.1]
Leaf nodes:                     Default
Sampling ratio:                 None
Classweight:                   None
Number of bins:                 [200]
Use fast heuristic:             True
Trim:                           False
Trim alpha:                     None
Classification objective:        None
Probability threshold:           None
K for precision:                 None
Loss function:                   Gbt.LossFunction.logistic
Smart search seed:               None
-----
```

`model.get_log()`

Use this function to retrieve a log file generated during model training.

Usage

```
model_object.get_log()
```

Parameters

- *model_object* - Specify the model whose log you want to retrieve

Example

This example below specifies to retrieve the log file generated during training of the GbtModel model. See [skytree.prediction.learn\(\)](#) (page 82) for information on how this model was trained.

Get the log for the GbtModel:

```
GbtModel.get_log()
```

Print the log to your screen:

```
print GbtModel.get_log().getvalue()
```

Write the log to a file:

```
with open('/Users/<username>/<path>/mylogs.txt','w') as f:  
    f.write(model.get_log().getvalue())
```

`model.get_pdp_variable_names()`

Use this function to retrieve a list of partial dependency plot variables generated during a model run.

Usage

```
pdp_variable_object = model_object.get_pdp_variable_names()
```

Parameters

- `pdp_variable_object` - Specify a variable that will contain the retrieved partial dependency variable object
- `model_object` - Specify the model object that will contain the dataset

Example

The following example retrieves the partial dependency plot variables from a model object called `model`. The PDP variables are then stored in a `pdps` object.

```
pdps = model.get_pdp_variable_names()
```

You can then run the following command to view the list of PDP variables and variable pairs (if applicable).

```
for variable_names in model.get_pdp_variable_names():  
    print variable_names
```

The result would be similar to below.

```
[u'capital-gain']  
[u'education-num']  
[u'capital-loss']  
[u'age']  
[u'hours-per-week']  
[u'relationship']  
[u'occupation']  
[u'marital-status']  
[u'education']  
[u'native-country']  
[u'capital-gain', u'education-num']  
[u'capital-gain', u'capital-loss']  
[u'capital-gain', u'age']  
[u'capital-gain', u'hours-per-week']  
[u'capital-gain', u'relationship']  
[u'capital-gain', u'occupation']  
[u'capital-gain', u'marital-status']  
[u'capital-gain', u'education']  
[u'capital-gain', u'native-country']  
[u'education-num', u'capital-loss']  
[u'education-num', u'age']  
[u'education-num', u'hours-per-week']  
[u'education-num', u'relationship']  
[u'education-num', u'occupation']  
[u'education-num', u'marital-status']  
[u'education-num', u'education']  
[u'education-num', u'native-country']  
[u'capital-loss', u'age']  
[u'capital-loss', u'hours-per-week']  
[u'capital-loss', u'relationship']
```

```
[u'capital-loss', u'occupation']
[u'capital-loss', u'marital-status']
[u'capital-loss', u'education']
[u'capital-loss', u'native-country']
[u'age', u'hours-per-week']
[u'age', u'relationship']
[u'age', u'occupation']
[u'age', u'marital-status']
[u'age', u'education']
[u'age', u'native-country']
[u'hours-per-week', u'relationship']
[u'hours-per-week', u'occupation']
[u'hours-per-week', u'marital-status']
[u'hours-per-week', u'education']
[u'hours-per-week', u'native-country']
[u'relationship', u'occupation']
[u'relationship', u'marital-status']
[u'relationship', u'education']
[u'relationship', u'native-country']
[u'occupation', u'marital-status']
[u'occupation', u'education']
[u'occupation', u'native-country']
[u'marital-status', u'education']
[u'marital-status', u'native-country']
[u'education', u'native-country']
```

`model.get_pmml_gzip()`

If models are trained with the `enable_pmml_out=True` configuration option, then those models can be exported into PMML (Predictive Model Markup Language) format, allowing the use of an external code for scoring. The output produces a gzip file.

For more information on PMML format, please refer to the PMML 4.2 section on the [Data Mining Group Web site](#). Also, refer to the PMML Usage Restrictions section following the example for important usage information.

Usage

```
model_object.get_pmml_gzip()
```

Parameters

- *model_object* - Specify the model whose PMML output you want to retrieve. Note that the model must have been trained with `enable_pmml_out = True` as part of the configuration.

Example

This example below creates a GBTR model that specifies PMML output. The `get_pmml_gzip()` function is then called to retrieve the output.

Create the configuration:

```
configuration2 = GbtrConfig()
configuration2.num_trees = [4]
configuration2.holdout_ratio = 0.2
configuration2.enable_pmml_out = True
```

Train the model:

```
GbtrModel = skytree.prediction.learn(
    incomeid, "id", configuration2, name="GbtrPmmlModel"
)
```

Get the PMML Output:

```
GbtrModel.get_pmml_gzip()
```

PMML Usage Restrictions

• RDF/RDFR Restrictions

RDF/RDFR resulting models use confidence rather than probability to express each leaf's `ScoreDistribution` elements. Confidence is different from probability in that the confidences of all classes do not necessarily sum to one, and in fact nodes can have confidence values that are greater than one.

Given a query point, prediction is performed by, for each class, summing the confidences at the leaf nodes encountered by the query point in all the trees. Then, the posterior probability of a given class is found by dividing that class' total confidence by the sum of all classes' confidences. Alternately, the class with the largest overall confidence may be taken as the query point's predicted class.

• GBT/GBTR Restrictions

In order to compute scores for the model, scores of all the trees in the model have to be summed. A function is then applied to the resulting sum. The function is provided under the node

```
<PostProcessor>
<![CDATA[(1/(1 + e^(-2*s)))]>
</PostProcessor>
```

in the PMML file itself. The symbol *s* in the function refers to the sum of the scores of all the trees.

All options available for GBT are available for PMML output except for ranking functions (i.e., if a loss function is specified as other than `logistic` (default)).

All options available for GBTR are available for PMML output.

- **Ensemble GBT Restrictions**

Because Ensemble GBT applies the concept of “bagging”, the resulting models have sets of GBTs. Unfortunately, the standard PMML format does not currently support the concept of multiple models with a depth of hierarchy greater than one. In order to overcome this limitation we introduce custom PMML to correctly represent that model.

In the PMML export of Ensemble GBT, we embed a complete GBT within each `<Segment>` by introducing another level of `<Segmentation>`. Each GBT within a `<Segment>` block has its own `<PostProcessor>`, which is used to compute result of that ensemble. The results of all GBTs can then be averaged to get the final answer.

- **SVM**

PMML Export is not currently available for SVM models.

`models.visualize_pdp()`

Use the `visualize_pdp` function to create plots that show the dependence between a feature or set of features marginalized over all other features.

Usage

```
visualize_object = model_object.visualize_pdp(name, column_name1, column_name2).ready()
```

Parameters

- *visualize_object* - Specify a variable that will contain the retrieved partial dependency variable object
- *model_object* - Specify the model object that will contain the PDPs
- *name* - A string containing the name to assign to this plot
- *column_name1, column_name2* - Specify the column name or names to be included in the plot as the features. No more than two column names can be referenced.

Examples

Example 1

The following example creates a plot from a model. Because this plot is made from a model, a column name must be specified. In the example below, the `capital-gain` column is used.

```
visualize = model.visualize_pdp(  
    "Model visualization Plot",  
    "capital-gain"  
) .ready()
```

Example 2

The following example also creates a plot from a model. This plot is generated based on both the `marital-status` (X axis) and `education` (Y axis) columns.

```
visualize = model.visualize_pdp(  
    "Model visualization Plot",  
    "marital-status",  
    "education"  
) .ready()
```

`model.test()`

In the training stage of machine learning, labeled data is used to generate a model. The testing (or evaluation) stage of machine learning evaluates the model against data that was not presented during the training stage. If the new data is labeled, then Skytree can generate a score that measures the generalization power of the model.

This function tests a model against a dataset that includes a unique ID column.

Note: When using a model to predict labels or targets for new data, any new levels of a categorical variable are treated as missing values by the machine learning model.

Usage

```
test_object = model_object.test(  
    dataset_object,  
    name=result_name,  
    score_config=score_config_object  
) .ready()
```

Parameters

- `test_object` - Specify a variable to contain the test result
- `dataset_object` - Specify the dataset to test the model on
- `name=result_name` - Optionally specify a string to use as the name for this test result
- `score_config=score_config_object` - Optionally specify a ScoreConfiguration object, which controls the model evaluation. The ScoreConfiguration object can include the following options:
 - `quantiles` - Optionally specify to use a specific number of (equidistant) quantiles when computing capture deviation. This value defaults to 10.
 - `class_weights` - Optionally specify a list of class weights to use during scoring. When specified, be sure to include one class weight for each label class.
 - `k_for_precision` - Optionally specify *k* as a value between 0 and 1 for precision and recall at the top 100k-th percentile. *k* defaults to 0.1, computing precision and recall at the top 10-th percentile.

Examples

Example 1

The following example tests an AutoModel model in the predict project. The ID for this model is 1908052802213767514. This model is based on the `income.data.ids` dataset, whose ID is 8483059517210011871. This dataset includes an ID column. Datasets that do not have an ID column cannot be provided in a test.

Get the income.data dataset:

```
incomeid = predict.get_dataset("8483059517210011871")
```

Get the automodel Model:

```
automodel = predict.get_model("1908052802213767514")
```

Test the model against the dataset:

```
test = automodel.test(incomeid, name="TestPredictOnly").ready()
```

Example 2

The following example performs a classification test. The model object referenced has an ID of 5796373431786034913.

Get the income.data dataset:

```
incomeid2 = predict.get_dataset("2922920584123794639")
```

Get the rdfmodel:

```
rdfmodel = predict.get_model("5796373431786034913")
```

Create a Score Configuration:

```
scoreconfig = ClassificationScoreConfig()  
scoreconfig.quantiles = 10
```

Test the model against the dataset:

```
rdftest = rdfmodel.test(  
    incomeid2, name="RDF Test", score_config=scoreconfig  
)
```

PlotRegistry()

Use the `PlotRegistry()` function to create a registry object. Once created, the object can be used to retrieve a list of resource entity types and IDs for test results and models in a project. Note that a *resource_id* is required when creating plots from test results.

Usage

```
plot_registry_entity_object = PlotRegistry(project_object)
```

Parameters

- *plot_registry_entity_object* - Specify a variable that will contain the retrieved registry types and IDs
- *project_object* - Specify the project object that will contain the registry types

Examples

The following example shows how to create a plot registry entity object. After this is created, the [findall\(\)](#) or [findById\(\)](#) functions can be used to retrieve the information that is required when creating plots for test results.

```
registry = PlotRegistry(project)
```

`plot_registry_entity.findall()`

Use the `findall` function to retrieve a list of resource entity types and IDs for test results and models in a project. Note that a `registry_id` is required when creating plots from test results.

Usage

```
resource_entity_object = registry_object.findall(type)
```

Parameters

- `resource_entity_object` - Specify a variable that will contain the retrieved registry types and IDs
- `registry_object` - Specify the registry object that contains the registry types
- `type` - Specify the a string for type of resource entity object to retrieve. The plot arguments will vary depending on the specified `type`. `type` can be one of the following:
 - DATASET
 - MODEL
 - TESTRESULT

Examples

Example 1

The following example shows how to retrieve the resource entity names and IDs for TESTRESULT types in project.

```
registry = PlotRegistry(project)
resource = registry.findall("TESTRESULT")
for registry in resource:
    print registry["name"]
    print registry["id"]
```

The output will be similar to below:

```
Capture Deviation Plot (Classification)
1084506945850907
Prediction Vs Actual (Regression)
5094586945860902
```

Example 2

The following example shows how to retrieve the resource entity names and IDs for MODEL types in project.

```
registry = PlotRegistry(project)
resource = registry.findall("MODEL")
for registry in resource:
    print registry["name"]
    print registry["id"]
```

The output will be similar to below:

```
PDP_1D_v1
4543534123453455
PDP_2D_v1
454347897549385455
```

`plot_registry_entity.findById()`

Use the `findById` function to retrieve a list of plot arguments to use when generating plots for test results. These plot arguments are required when generating plots for test results.

Usage

```
list_of_registry_entities = plot_registry_entity_object.findById("id")
```

Parameters

- `list_of_registry_entities` - Specify a variable that will contain the retrieved registry types and IDs
- `plot_registry_entity_object` - Specify the registry object that contains the registry types
- `id` - Specify the a string for the ID of the desired result registry type. Valid IDs are obtained by running [findall\(\)](#).

Examples

The following example retrieves plot arguments for a classification resource. The ID for classification resources is 1084506945850907. Specify this value to retrieve a list of plot arguments that can be included in the plot.

Retrieve the Resource ID

```
registry = PlotRegistry(project)
resource_id = registry.findall("TESTRESULT")
for registry in resource_id:
    print registry["name"]
    print registry["id"]
```

The output will be similar to below:

```
Capture Deviation Plot (Classification)
1084506945850907
Prediction Vs Actual (Regression)
5094586945860902
```

Retrieve the Plot Arguments for Classification Plots

```
registry = PlotRegistry(project)
plot_args = registry.findById("1084506945850907")
print plot_args["argsSpec"]
```

The output will be similar to below. These are the arguments that should be included in the Capture Deviation plot.

```
{
  "columns": ["Objective Column", "Predicted Probabilities", "Predicted Labels",
    "Predicted Categories"],
  "summarizer": [
    {
      "name": "numBuckets",
      "kind": "optional",
      "type": "integer",
      "default": 10
    }
  ],
  "plot": []
}
```

`plot.terminate()`

You may find it necessary to cancel a plot that is currently INPROGRESS. The terminate function can be used to stop the creation process. Note that you cannot interrupt plots that have a status of READY.

Usage

```
plot_object.terminate()
```

Parameters

- `plot_object` - Specify the plot to interrupt

Example

The following example terminates a visualize plot that is currently being created.

```
visualize.terminate()
```

The response returns True or False, depending on whether the terminate was successful.

`test.create_probabilities_dataset()`

This function creates a dataset from the probabilities data in a classification test result.

Usage

```
dataset_object = test_object.create_probabilities_dataset(  
    name=dataset_name  
) .ready()
```

Parameters

- *dataset_object* - Specify the dataset object that will contain the new dataset
- *test_object* - Specify the test object that contains the data for the new dataset. This must contain a classification test result.
- *name=dataset_name* - A string containing the name to assign to this dataset (optional)

Example

The following example retrieves a Classification result file in the predict project. The ID for this file is 8443502907664023107. The result is then referenced in a new dataset that is built from the probabilities in this result.

```
PredictResult = predict.get_result("8534304197005081735")  
newdatasetresult = PredictResult.create_probabilities_dataset(  
    name="DatasetFromProbabilities"  
) .ready()
```

`test.create_labels_dataset()`

This function creates a dataset from the labels data in a classification test result. The resulting dataset includes three columns:

- The ID
- The label (-1,1 for binary classification or 0 to N-1 for n-category classification)
- The labels/categories used in the training dataset

Usage

```
dataset_object = test_object.create_labels_dataset(  
    name=dataset_name  
) .ready()
```

Parameters

- `dataset_object` - Specify the dataset object that will contain the new dataset
- `test_object` - Specify the test object that contains the data for the new dataset. This must contain a classification test result.
- `name=dataset_name` - A string containing the name to assign to this dataset (optional)

Example

The following example retrieves a Classification result file in the Predict project. The ID for this file is 8443502907664023107. The result is then referenced in a new dataset that is built from the labels in this result.

```
labelsresult = predict.get_result("8534304197005081735")  
new_dataset_labels = labelsresult.create_labels_dataset(  
    name="DatasetFromLabels"  
) .ready()
```

This command produces two datasets: one that includes a third column for the categories and one that does not.

Run the following to view the new dataset, including the category information.

```
print new_dataset_labels.summary()
```

```
Skytree dataset summary  
-----  
id:                3797725121176472705  
name:              DatasetFromLabels with labels  
created at:        2015-10-27 09:28:11  
updated at:        2015-10-27 09:28:32  
source id:         None  
state:             Sample  
type:              Transformed  
number of rows:    None  
number of models:  0  
-----  
status:  
message: SAMPLED: Data Set has been validated and is ready to use  
code: READY  
-----  
Transform:
```

```
-----  
AddCategoricalNameColumn: map {u'1': u'>50K', u'-1': u'<=50K'}  
-----  
-----
```

test.create_targets_dataset()

This function creates a dataset from the targets in a regression test result.

Usage

```
dataset_object = test_object.create_targets_dataset(  
    name=dataset_name  
) .ready()
```

Parameters

- *dataset_object* - Specify the dataset object that will contain the new dataset
- *test_object* - Specify the test that contains the data for the new dataset. This must contain a regression test result.
- *name=dataset_name* - A string containing the name to assign to this dataset (optional)

Example

The following example retrieves a Regression result file in the Predict project. The ID for this file is 8443502907664023107. The result is then referenced in a new dataset that is built from the targets in this result.

```
targetssresult = predict.get_result("1908052802213767514")  
newdatasettargets = labels.create_targets_dataset(  
    name="DatasetFromTargets"  
) .ready()
```

`test.summary()`

Use `summary` with any READY test result to output a summary of the results

Usage

```
print test_object.summary()
```

Parameters

- `test_object` - The test result that you want to retrieve

Example

The example gets a test result whose ID is 79494999998156928697. This test result is in the `predict` project. The test result object is `gbtresult`.

```
gbtresult = predict.get_result("79494999998156928697")
print gbtresult.summary()
```

The output varies based on whether the result is for a classification problem or a regression problem. For a classification test result, the summary will be similar to the following:

```
Skytree classification test result
-----
False positive rate: 0.0600794
Precision:           0.767109
Recall:              0.635884
Accuracy:            0.867759
F score:             0.695359
Confusion matrix:
---
Class -1: Wrong: 439. Right: 6868. Total 7307.
Class 1: Wrong: 828. Right: 1446. Total 2274.
Overall : Wrong: 1267. Right: 8314. Total 9581.
---
Gini:                0.844444
Capture Deviation:   0.0409448
-----
```

test.**terminate()**

You may find it necessary to cancel a test that is currently INPROGRESS. The `terminate` function can be used to stop the creation process. Note that you cannot interrupt test results that have a status of READY.

Usage

```
test_object.terminate()
```

Parameters

- *test_object* - Specify the test that you want to interrupt

Example

The following example terminates an RDF test that is being generated.

```
rdftest.terminate()
```

The response returns True or False, depending on whether the terminate was successful.

`test.visualize()`

Use the `visualize` function to create a plot that shows the relation between predictors and outcomes (regression plot) or to visualize the capture deviation (classification plot).

Usage

```
visualize_object = result_object.visualize(plot_args, name, resource_id).ready()
```

Parameters

- *visualize_object* - Specify a variable that will contain the retrieved partial dependency variable object
- *result_object* - Specify the result object that will contain the plot
- *plot_args* - Specify the list of plot arguments to include in the plot. These are retrieved using `findById()`. Plot arguments can include the following.

- Columns.

Columns for Classification results can include Objective Column, Predicted Labels, Predicted Probabilities, Predicted Categories

Columns for Regression results can include Objective Column and Predicted Targets

- Summarizer Arguments.

```
"name" : "argument name"
```

```
"type" : "boolean | integer | double | string | list<integer> | list<double> | list<string>"
```

```
"kind" : "required | optional | fixed | partial" //partial can only apply to list types
```

```
"default" : <default value> // optional unless kind is fixed
```

```
"min_size" : <int> // required only for list types. Can be set to 0.
```

```
"max_size" : <int> // required only for list types. Unlimited size if not provided.
```

- *name* - A string containing the name to assign to this plot
- *resource_id* - The ID of the resource that is used for this plot. These are retrieved using `findAll()`.

Examples

Example 1

The following example generates a Capture Deviation plot for a classification test result that resides in the project project. The plot is based on the objective column and will include the predicted labels, probabilities, and categories.

Determine the resource IDs in project

```
registry = PlotRegistry(project)
resource_id = registry.findAll("TESTRESULT")
for registry in resource_id:
    print registry["name"]
    print registry["id"]
```

The output will be similar to below:

```
Capture Deviation Plot (Classification)
1084506945850907
Prediction Vs Actual (Regression)
5094586945860902
```

Determine the plot args in a test result

Because the plot will be based on a classification result, the resource ID used will be 1084506945850907. Specify this value to retrieve a list of plot arguments that can be included in the plot.

```
registry = PlotRegistry(project)
plot_args = registry.findById("1084506945850907")
print plot_args["argsSpec"]
```

The output will be similar to below. These are the arguments that should be included in the Capture Deviation plot.

```
{
  "columns": ["Objective Column", "Predicted Probabilities", "Predicted Labels",
    "Predicted Categories"],
  "summarizer": [
    {
      "name": "numBuckets",
      "kind": "optional",
      "type": "integer",
      "default": 10
    }
  ],
  "plot": []
}
```

Specify the arguments to include in the Capture Deviation plot

```
plot_args = "{ \"columns\": [\"Objective Column\", \"Predicted Probabilities\",
  \"Predicted Labels\", \"Predicted Categories\"],
  \"summarizer\": [ { \"name\": \"numBuckets\", \"kind\": \"fixed\",
    \"type\": \"integer\", \"default\": 10 } ], \"plot\": [] }";
plot_name = "Capture Deviation"
plot_registry_id = "1084506945850907"
```

Generate the plot

```
plot = result.visualize(plot_args, plot_name, plot_registry_id).ready()
```

Example 2

The following example generates an Actual vs. Prediction plot for a regression test result. The plot is based on the objective column and will include the predicted targets.

Determine the resource IDs in project

```
registry = PlotRegistry(project)
resource_id = registry.findAll("TESTRESULT")
for registry in resource_id:
  print registry["name"]
  print registry["id"]
```

The output will be similar to below:


```
Capture Deviation Plot (Classification)
1084506945850907
Prediction Vs Actual (Regression)
5094586945860902
```

Determine the Plot Arguments in a test result

Because the plot will be based on a regression result, the resource ID used will be 5094586945860902. Specify this value to retrieve a list of plot arguments that can be included in the plot.

```
resource = PlotRegistry(project)
plot_args = resource.findById("5094586945860902")
print plot_args["argsSpec"]
```

The output will be similar to below. These are the arguments that should be included in the Prediction vs. Actual plot.

```
{
  "columns": ["Objective Column", "Predicted Targets"],
  "summarizer": [
    {
      "name": "returnSize",
      "kind": "fixed",
      "type": "integer",
      "default": 10000
    }
  ],
  "plot": []
}
```

Specify the arguments to include in the Prediction vs. Actual plot

```
plot_args = "{ \"columns\": [\"Objective Column\", \"Predicted Targets\"],
  \"summarizer\": [ { \"name\": \"returnSize\", \"kind\": \"fixed\",
    \"type\": \"integer\", \"default\": 10000 } ], \"plot\": [] }";
plot_name = "Actual vs. Predict"
plot_registry_id = "5094586945860902"
```

Generate the plot

```
plot = test_result.visualize(plot_args, plot_name, plot_registry_id).ready()
```

test.get_log()

Use this function to retrieve a log file generated during a test.

Usage

```
test_object.get_log()
```

Parameters

- *test_object* - Specify the test whose log you want to retrieve

Example

This example retrieves the log from the `rdftest` from Example 2 in the [skytree.prediction.test\(\)](#) section.

Print the log to your screen:

```
print rdftest.get_log().getvalue()
```

Write the log to a file:

```
with open('/Users/<username>/<path>/mylogs.txt', 'w') as f:  
    f.write(rdftest.get_log().getvalue())
```

`test.get_probabilities()`

Use this function to retrieve the stored computed probabilities from a classification test set. Note that new datasets can be created from the probabilities in a test result. Refer to [test.create_probabilities_dataset\(\)](#) (page 99) for more information.

Usage

```
probabilities_object = test_object.get_probabilities()
```

Parameters

- *probabilities_object* - Specify a variable to store the probabilities. This is returned as a generator. It can be used to iterate through the predictions, one for each row in the test dataset, and each returned item is a tuple of the ID and probability.
- *test_object* - Specify the test that contains the probabilities you want to retrieve. This test must contain a classification test result.

Example

This example below specifies to retrieve the probabilities generated from a GBT test in the predict project. It then reshapes the output into a list of (id, probability) pairs.

Get the result:

```
gbtresult = predict.get_result("586438935509143969")
```

Get the Probabilities from this result:

```
probs = gbtresult.get_probabilities()
```

Print the Probabilities:

```
for (id, probability) in gbtresult.get_probabilities():  
    print (id, probability)
```

`test.get_labels()`

When generating a labels file, the labels are always "-1" and "1" for a binary classification problem. If the original dataset has a more descriptive label, such as "<=50k" and ">50k" (as is the case with the `income.data` file), then Skytree creates a mapping from the original labels to the integers. It does this via lexicographically ordering the label column. The first label gets mapped to "-1" and the second label gets mapped to "1" in the case of binary classification.

Use the `get_labels` function to retrieve the stored computed labels from a classification test set. Note that new datasets can be created from the labels in a test result. Refer to [test.create_labels_dataset\(\)](#) (page 100) for more information.

Usage

```
labels_object = test_object.get_labels()
```

Parameters

- *labels_object* - Specify a variable to store the labels. This is returned as a generator. It can be used to iterate through the predictions, one for each row in the test dataset, and each returned item is a tuple of the ID and label.
- *test_object* - Specify the test that contains the labels you want to retrieve. This test must contain a classification test result.

Example

This example below specifies to retrieve the targets generated from a GBT test in the predict project. It then reshapes the output into a list of (id, label) pairs.

Get the Classification result:

```
gbtresult = predict.get_result("586438935509143969")
```

Get the Labels from this result:

```
labels = gbtresult.get_labels()
```

Print the Labels:

```
for (id, label) in gbtresult.get_labels():  
    print (id, label)
```

`test.get_targets()`

Use this function to retrieve the output that includes the stored computed targets for the regression model. Note that new datasets can be created from the targets in a test result. Refer to [test.create_targets_dataset\(\)](#) (page 102) for more information.

Usage

```
target_object = test_object.get_targets()
```

Parameters

- *target_object* - Specify a variable to store the targets. This is returned as a generator. It can be used to iterate through the predictions, one for each row in the test dataset, and each returned item is a tuple of the ID and target.
- *test_object* - Specify the test that contains the targets you want to retrieve. This test must contain a regression test result.

Example

This example below specifies to retrieve the targets generated from a GBTR test in the predict project. It then reshapes the output into a list of (id, target) pairs.

Get the Regression result:

```
gbtrresult = predict.get_result("8443502907664023107")
```

Get the Targets and Target Values from this result:

```
targets = gbtrresult.get_targets()
```

Print the Targets:

```
for (id, target) in gbtrresults.get_targets():  
    print (id, target)
```


Appendix A Models

Skytree allows the user to train a variety of different model types on their datasets, tune the parameters of those models, test those models, and use them to predict values.

Classification vs. Regression

Skytree methods allow you to perform classification and regression problems.

Binary Classification

Given two sets of points, one for class A and one for class B, users want to find a model that can separate efficiently the points of classes A and B. Classification models are trained on a dataset where the correct class labels are known. They are used to predict, for unlabeled data, the probabilities that each point is of class A or B, and therefore predict a class label.

Regression

Regression problems involve predicting a continuous value rather than a categorical class label. The training dataset needs to have correct objective values for each rows, and the resulting model predicts, for unlabeled data, the value of the objective.

Methods

Skytree allows you to build models using one of the following learning machine learning methods:

- AutoModel
- Gradient Boosted Trees (GBT)
- Gradient Boosted Trees Regression (GBTR)
- Generalized Linear Models Classification (GLMC)
- Generalized Linear Models Regression (GLMR)
- Random Decision Forests (RDF)
- Random Decision Forests Regression (RDFR)
- Support Vector Machines (SVM)

These methods are summarized in the sections that follow.

AutoModel Models

Skytree AutoModel automatically determines the best model and parameter settings to solve a particular problem. It explores, in an intelligent way, all possible models and their parameter settings and picks the model that is most effective at making the required prediction. AutoModel selects whether to use classification or regression based on the type of the objective column; continuous values are predicted with regression models, and categorical values are predicted with classification models.

GBT Models

Gradient Boosted Trees (GBT) is a machine learning algorithm combining decision trees with boosting. The latter is a numerical gradient optimization method that minimizes a loss function, in this case, the deviance.

The optimization is achieved by successively adding trees that best reduce the loss function. The first tree reduces the loss function to the greatest extent possible, while subsequent trees focus on a residual that represents the poorest responses of the model.

As the boosting iteration proceeds, the algorithm adjusts the relative weighting of each tree without modifying the previous trees themselves. The resulting model is a linear combination of all trees.

GBTR Models

Gradient Boosted Trees Regression (GBTR) is a machine learning algorithm combining regression trees with boosting. The latter is a numerical gradient optimization method that minimizes a loss function, in this case, the deviance.

The optimization is achieved by successively adding trees that best reduce the loss function. The first tree reduces the loss function to the greatest extent possible, while subsequent trees focus on a residual that represents the poorest responses of the model.

As the boosting iteration proceeds, the algorithm adjusts the relative weighting of each tree without modifying the previous trees themselves. The resulting model is a linear combination of all trees.

Generalized Linear Models - Classification and Regression

Linear Regression models the relationship between a group of regressors (predictors) $X_1, \dots, X_p$ and the prediction values y using a linear fit. The Generalized Linear Model method is an extension of ordinary Linear Regression and consists of the following components:

- A probability distribution family f in the exponential family with expected value $\mu = E_f(y)$
- A linear model $\eta = \mathbf{x}^T \beta$
- A link function g relating $g(\mu) = \eta$

Accordingly, in order to specify a GLM problem, you must choose family function f , link function g , and any parameters needed to train the linear model weights β .

Note: A family function cannot be specified in GLMC because only a single family (Binomial) is available.

Skytree supports the following Family/Link Function combinations:

Table 9: Family/Link Function Combinations

Family	Link Function					
	Identity	Square Root	Inverse	Log	Logit	Cloglog
Gaussian	X					
Poisson	X	X		X		
Gamma			X	X		
Tweedie				X		
Binomial					X	X

Note: Certain link functions, such as Log and Square-Root, are only applicable to non-negative responses.

RDF Models

The Random Decision Forests (RDF) algorithm is an ensemble classifier composed of a collection of decision trees. The ensemble returns classification based on the aggregate results of the individual trees. Each tree in the forest is a weak classifier, trained by selecting a subset of the training data, usually with replacement, and a random selection of the available features.

Each tree is grown using a sampling of the original training data. Nodes of the tree are split by considering a subset of all features of the data and are grown without pruning.

Each ensemble classification requires classification by each tree in the forest. The class with a plurality of votes determines the final classification.

Several factors affect the error of the model. Strong individual trees in the ensemble decrease the error rate and occur with a larger number of trees. However, a large number of trees also introduces correlated trees which increase the error rate. For these reasons, among others, it's important that the parameters of the Random Decision Forest be tuned to provide the best possible error rate.

RDFR Models

The Random Decision Forest Regression (RDFR) algorithm is an ensemble regression model comprising a collection of regression trees. The ensemble returns predictions based on the average of the results of the individual trees. Each tree in the forest is a weak regression model, trained by selecting a subset of the training data, usually with replacement, and a random selection of the available features.

Each tree is grown using a sampling of the original training data. Nodes of the tree are split by considering a subset of all features of the data and are grown without pruning.

Each ensemble regression prediction requires a regression prediction by each tree in the forest. The average of the predictions of the individual trees determines the final prediction.

Several factors affect the error of the model. Strong individual trees in the ensemble decrease the error rate and occur with a larger number of trees. However, a large number of trees also introduces correlated trees which increase the error rate. For these reasons, among others, it's important that the parameters of the Random Decision Forest Regression model be tuned to provide the best possible error rate.

SVM Models

The [Support Vector Machine](http://en.wikipedia.org/wiki/Support_vector_machine) (http://en.wikipedia.org/wiki/Support_vector_machine) (SVM) is one of the most successful classification algorithms. It is based on the revolutionary theory of kernel learning. The main concept is that SVM tries to fit a hyperplane that optimally separates different classes of data. The hyperplane can be linear or nonlinear.

- Find the best boundary to separate points into two classes
- Maps points onto a high-dimensional space so that it can learn complex nonlinear decision boundaries
- If no perfect separation, maximizes the margin
- An advantage is no local-minimum situations
- Naively scales as N^2 - N^3 ; Skytree scaling is data dependent

Model Configuration

The following sections describe the configuration options that can be used in each method when training models.

- [AutoModel Configuration](#) (page 118)
- [GBT Model Configuration](#) (page 119)
- [GBTR Model Configuration](#) (page 124)
- [GLMC Model Configuration](#) (page 128)
- [GLMR Model Configuration](#) (page 131)
- [RDF Model Configuration](#) (page 134)
- [RDFR Model Configuration](#) (page 137)
- [SVM Model Configuration](#) (page 139)

AutoModel Configuration

An AutoModel configuration can be specified when training models. The following options can be specified in an AutoModel configuration.

Table 10: AutoModel Model Validation Parameters

Parameter	Description
holdout_ratio	Specify a decimal value for the fraction of the table to be held out for tuning. If this option is specified, a collection of models will be built and evaluated on the held-out tuning set, then the parameters from the best model will be used to train a model on the entire training dataset.
holdout_seed	Optionally specify a long value for the holdout sampling random number seed. This value also serves as the seed for num_folds. If omitted, a time-based seed will be used.
num_folds	Specify an integer for the number of folds for k-fold cross-validation. If holdout_ratio is also specified, then Monte Carlo cross-validation (with randomly drawn holdout sets) is instead performed with the specified number of restarts. If this option is specified, a collection of models will be built and evaluated, then the parameters from the best model will be used to train a model on the entire training dataset.

Table 11: Non-Tunable AutoModel Parameters

Parameter	Description
classification_objective	Optionally specify either automodel.classification_objective.fscore (default) or automodel.classification_objective.accuracy as the objective for classification threshold tuning.
enable_partial_dependencies	Specify whether to export the partial dependencies of the relevant features and/or feature pairs in JSON format. These can be used to generate partial dependence plots.
enable_pmm1_out	Specify whether to export trained models into PMML (Predictive Model Markup Language) format. This allows the use of an external code for scoring.
k_for_precision	Specify k as a decimal value between 0 and 1 for precision at the top 100 k -th percentile. This value defaults to 0.1.

Table 11: Non-Tunable AutoModel Parameters (continued)

Parameter	Description
limit_parameters	This option restricts the parameter space (for example, in num_trees, tree_depth, etc.) in order to reduce the training time. This option is enabled by default. Note that when GBT or GBTR is the method used to generate the model, limit_parameters will set the following upper bound values: • num_trees <= 3000 • tree_depth <= 9 • max_splits <= 256
smart_search_iterations	Optionally specify the number of search rounds to try for tuning. This value must be greater than 0 and defaults to 100.
smart_search_seed	Optionally specify the search seed. If omitted, a time-based seed will be used.
testing_objective	Specify one of the following objectives for selecting the test model from a set of tuned models. Classification problem Options • automodel.testing_objective.gini (default for Classification) • automodel.testing_objective.fscore • automodel.testing_objective.accuracy • automodel.testing_objective.capture_dev • automodel.testing_objective.precision_at_k Regression problem options • automodel.testing_objective.normalized_gini • automodel.testing_objective.mean_absolute_error (default for Regression) • automodel.testing_objective.mean_squared_error • automodel.testing_objective.coeff_determination

GBT Model Configuration

A GBT model configuration can be specified when training, tuning, and testing models. The following options can be specified in a GBT configuration.

Table 12: GBT Model Validation Parameters

Parameter	Description
holdout_ratio	Specify a decimal value for the fraction of the table to be held out for tuning. If this option is specified, a collection of models will be built and evaluated on the held-out tuning set, then the parameters from the best model will be used to train a model on the entire training dataset.
holdout_seed	Optionally specify a long value for the holdout sampling random number seed. This value also serves as the seed for num_folds. If omitted, a time-based seed will be used.
num_folds	Specify an integer for the number of folds for k-fold cross-validation. If holdout_ratio is also specified, then Monte Carlo cross-validation (with randomly drawn holdout sets) is instead performed with the specified number of restarts. If this option is specified, a collection of models will be built and evaluated, then the parameters from the best model will be used to train a model on the entire training dataset.
smart_search	Specify to use an intelligent tuning technique (instead of naive grid search). When this flag is set to True, no other tuning parameters are required; only model validation options are expected. GBT will automatically search for the best parameter setting. Because Smart Search performs intelligent tuning, this option cannot be used with probability_threshold.
smart_search_iterations	If smart_search is enabled, optionally specify the number of search rounds to try for tuning. This value must be greater than 0 and defaults to 100.

Table 13: Tunable GBT Parameters specified as a list. When doing a grid search, all possible combinations of values are tried.

Parameter	Description
ensemble_size	Specify a list of integers. The ensemble GBT algorithm applies the concept of bootstrap aggregation, or “bagging”, to GBTs. The training table is sampled a number of times, and a new GBT is built for each sample. This option specifies the number of GBTs to build.
imbalance_scale	Specify a list of decimal values for the relative size of all classes with respect to the smallest class. This option is only available if imbalance is enabled.
learning_rate	Specify a list of decimal values value for the learning rate for each iteration. This value defaults to 0.1 and must be > 0 and <= 1.

Table 13: Tunable GBT Parameters specified as a list. When doing a grid search, all possible combinations of values are tried. (continued)

Parameter	Description
max_splits	Specify the number of splits in the tree. This value must be ≥ 0. A value of 0 disables this option. Either this value or min_node_weight must be > 0 if tree_depth is explicitly set to 0. If both tree_depth and max_splits are > 0, then Skytree finds the best splits that exist within the depth level, up to max_splits.
min_node_weight	Specify an array for the minimum bound on the total size in each leaf during tree building. Either this value or max_splits must be > 0 if tree_depth is explicitly set to 0.
num_dimensions	Specify a list of integers for the number of dimensions to sample at each tree node. Only used in conjunction with ensemble_size. If omitted, all dimensions are used.
num_trees	This is required except when using smart search. In that case, this is optional. Specify a list of numbers to indicate the number of trees in the ensemble. This value must be > 0.
regularization_bins	Specify a list of integers for the number of bins to use per dimension/attribute of the data. This option is only used with regularization. If regularization is enabled, then this value defaults to 200.
sampling_ratio	Specify a list of doubles to indicate the per-class sampling ratio. You can use a single sampling_ratio that will be used for each class, or specify sampling_ratio values for each class. If a value is not specified and sample_with_replacement is enabled, then full bootstrapping will be used.
tree_depth	Specify a list of integers for the depth to which each ensemble is built. If set to 0, trees will be built to the fullest extent. In this case, then you must also specify a positive value for either min_node_weight or max_splits. If this value is not set, and values for max_splits and min_node_weight are likewise not set, then this value defaults to 3. If this value and max_splits are both > 0, then Skytree finds the best splits that exist within the depth level, up to max_splits.

Table 14: Non-Tunable GBT Parameters

Parameter	Description
<code>cardinality_based_dimension_sampling</code>	Categorical dimensions with larger number of unique values in a given node will have a proportionately higher chance of being selected when sampling dimensions. Continuous features are treated as having 1 unique value. When <code>False</code> , all dimensions have an equal chance of being selected in dimension sampling.
<code>classification_objective</code>	Optionally specify either <code>"gbt.classification_objective.fscore"</code> (default) or <code>"gbt.classification_objective.accuracy"</code> as the objective for classification threshold tuning. This option cannot be used with <code>probability_threshold</code> .
<code>dimension_sampling_seed</code>	Specify a long value for the dimension sampling random number seed. If omitted, a time-based seed will be used.
<code>enable_partial_dependencies</code>	Specify whether to export the partial dependencies of the relevant features and/or feature pairs in JSON format. These can be used to generate partial dependence plots. This option is available when: • Training along with (model save OR testing) • Tuning followed by training + (model save OR testing)
<code>enable_pmm1_out</code>	Specify whether to export trained models into PMML (Predictive Model Markup Language) format. This allows the use of an external code for scoring.
<code>imbalance</code>	When enabled, the API will attempt to improve classification for imbalanced classes. Note that this option cannot be used with <code>sampling_ratio</code> .
<code>k_for_precision</code>	Specify <i>k</i> as a decimal value between 0 and 1 for precision at the top 100 <i>k</i> -th percentile. This value defaults to 0.1.
<code>loss_function</code>	Specify <code>gbt.loss_function.logistic</code> to use Logistic ranking.
<code>probability_threshold</code>	Specify a decimal value for the probability to be used as the threshold for classification. This option cannot be used with <code>classification_objective</code> or <code>testing_objective</code> . Similarly, because <code>probability_threshold</code> cannot be used during tuning, this option cannot be specified with <code>smart_search</code> .
<code>regularization</code>	Specify to use the fast decision tree construction heuristic. This option defaults to <code>True</code> for <code>gbt/gbtr</code> and defaults to <code>False</code> for ensemble <code>gbt/gbtr</code> . When regularization is enabled, <code>regularization_bins</code> defaults to 200.
<code>sample_with_replacement</code>	For each tree, sample the training points using replacement (bootstrap). This option is enabled by default. If set to <code>False</code> , then the <code>sampling_ratio</code> must be provided.

Table 14: Non-Tunable GBT Parameters (continued)

Parameter	Description
smart_search_seed	When smart_search is enabled, optionally specified a seed value for smart search. If omitted, a time-based seed will be used by default.
table_sampling_seed	Specify a long value for the data sampling random number seed. If omitted, a time-based seed will be used.
testing_objective	Specify one of the following objectives for selecting the test model from a set of tuned models. ▪ gbt.testing_objective.gini (default) ▪ gbt.testing_objective.fscore ▪ gbt.testing_objective.accuracy ▪ gbt.testing_objective.capture_dev ▪ gbt.testing_objective.precision_at_k This option cannot be used with probability_threshold.
trim	If enabled, observations with low importance in the model will be pruned in successive iterations. This can lead to significant speedup, but accuracy can suffer. This option defaults to False.
trim_alpha	Specify a decimal value > 0 and < 1. Higher values lead to more aggressive pruning of observations per iterations..

GBTR Model Configuration

A GBTR model configuration can be specified when training, tuning, and testing models. The following options can be specified in a GBTR configuration.

Table 15: GBTR Model Validation Parameters

Parameter	Description
holdout_ratio	Specify a decimal value for the fraction of the table to be held out for tuning. If this option is specified, a collection of models will be built and evaluated on the held-out tuning set, then the parameters from the best model will be used to train a model on the entire training dataset.
holdout_seed	Optionally specify a long value for the holdout sampling random number seed. This value also serves as the seed for num_folds. If omitted, a time-based seed will be used.
num_folds	Specify an integer for the number of folds for k-fold cross-validation. If holdout_ratio is also specified, then Monte Carlo cross-validation (with randomly drawn holdout sets) is instead performed with the specified number of restarts. If this option is specified, a collection of models will be built and evaluated, then the parameters from the best model will be used to train a model on the entire training dataset.
smart_search	Specify to use an intelligent tuning technique (instead of naive grid search). When this flag is set to True, no other tuning parameters are required; only model validation options are expected. GBTR will automatically search for the best parameter setting. Because Smart Search performs intelligent tuning, this option cannot be used with probability_threshold.
smart_search_iterations	If smart_search is enabled, optionally specify the number of search rounds to try for tuning. This value must be greater than 0 and defaults to 100.

Table 16: Tunable GBTR Parameters specified as a list. When doing a grid search, all possible combinations of values are tried.

Parameter	Description
ensemble_size	Specify a list of integers. The ensemble GBT algorithm applies the concept of bootstrap aggregation, or “bagging”, to GBTs. The training table is sampled a number of times, and a new GBT is built for each sample. This option specifies the number of GBTs to build.
learning_rate	Specify a list of decimal values value for the learning rate for each iteration. This value defaults to 0.1 and must be > 0 and <= 1.

Table 16: Tunable GBTR Parameters specified as a list. When doing a grid search, all possible combinations of values are tried. (continued)

Parameter	Description
log_clamp	When <code>loss_function</code> is <code>gbtr.loss_function.pslog</code> , <code>gbtr.loss_function.gmlog</code> , or <code>gbtr.loss_function.tdlog</code> , optionally specify the maximum absolute log value for any terminal node. This value must be > 0 .
max_splits	Specify the number of splits in the tree. This value must be ≥ 0 . A value of 0 disables this option. Either this value or <code>min_node_weight</code> must be > 0 if <code>tree_depth</code> is explicitly set to 0. If both <code>tree_depth</code> and <code>max_splits</code> are > 0 , then Skytree finds the best splits that exist within the depth level, up to <code>max_splits</code> .
min_node_weight	Specify an array for the minimum bound on the total size in each leaf during tree building. Either this value or <code>max_splits</code> must be > 0 if <code>max_tree_depth</code> is explicitly set to 0.
num_dimensions	Specify a list of integers for the number of dimensions to sample at each tree node. Only used in conjunction with <code>ensemble_size</code> . If omitted, all dimensions are used.
num_trees	This is required except when using smart search. In that case, this is optional. Specify a list of numbers to indicate the number of trees in the ensemble. This value must be > 0 .
regularization_bins	Specify a list of integers for the number of bins to use per dimension/attribute of the data. This option is only used with regularization. If regularization is enabled, then this value defaults to 200.
sampling_ratio	Specify a list of doubles to indicate the sampling ratio. If a value is not specified and <code>sample_with_replacement</code> is enabled, then full bootstrapping will be used.
tree_depth	Specify a list of integers for the depth to which each ensemble is built. If set to 0, trees will be built to the fullest extent. In this case, then you must also specify a positive value for either <code>min_node_weight</code> or <code>max_splits</code> . If this value is not set, and values for <code>max_splits</code> and <code>min_node_weight</code> are likewise not set, then this value defaults to 3. If this value and <code>max_splits</code> are both > 0 , then Skytree finds the best splits that exist within the depth level, up to <code>max_splits</code> .

Table 16: Tunable GBTR Parameters specified as a list. When doing a grid search, all possible combinations of values are tried. (continued)

Parameter	Description
tweedie_exponent	When <code>loss_function</code> is <code>gbtr.loss_function.tdlog</code> , specify a value for the exponent. This option has no default value and, therefore, must be explicitly specified when configuring the Tweedie loss function. This value must be > 1.0 . If users desire a value of 1, they should use Poisson distribution (<code>gbtr.loss_function.p slog.</code>). Similarly, if users desire an exponent equal to 2.0, they should specify Gamma distribution (<code>gbtr.loss_function.gmlog</code>).

Table 17: Non-Tunable GBTR Parameters

Parameter	Description
cardinality_based_dimension_sampling	Categorical dimensions with larger number of unique values in a given node will have a proportionately higher chance of being selected when sampling dimensions. Continuous features are treated as having 1 unique value. When set to <code>False</code> , all dimensions have an equal chance of being selected in dimension sampling
dimension_sampling_seed	Specify a long value for the dimension sampling random number seed. If omitted, a time-based seed will be used.
enable_partial_dependencies	Specify whether to export the partial dependencies of the relevant features and/or feature pairs in JSON format. These can be used to generate partial dependence plots. This option is available when: • Training along with (model save OR testing) • Tuning followed by training + (model save OR testing)
enable_pmm1_out	Specify whether to export trained models into PMML (Predictive Model Markup Language) format. This allows the use of an external code for scoring.
huber_loss_quantile	When <code>loss_function</code> is <code>gbtr.loss_function.huber</code> , specify the top percentile of error that should be considered as outliers. This value must be between 0 and 1.

Table 17: Non-Tunable GBTR Parameters (continued)

Parameter	Description
loss_function	Optionally specify a loss function method to use during regression. Available values include the following: <code>gbtr.loss_function.lad</code> - Perform least absolute deviation (LAD) regression <code>gbtr.loss_function.ls</code> - Perform least-squares regression <code>gbtr.loss_function.huber</code> - Perform Huber loss regression <code>gbtr.loss_function.psong</code> - Perform Poisson-log regression with positive integral targets (such as counts) <code>gbtr.loss_function.gmlog</code> - Perform Gamma-log regression with positive continuous targets <code>gbtr.loss_function.gminv</code> - Perform Gamma-inverse regression with positive continuous targets <code>gbtr.loss_function.tdlog</code> - Perform Tweedie-log regression If <code>gbtr.loss_function.huber</code> is specified, then you must also define a value for <code>huber_loss_quantile</code>. Similarly, if <code>gbtr.loss_function.tdlog</code> is specified, then you must also define a value for <code>tweedie_exponent</code>.
regularization	Specify to use the fast decision tree construction heuristic. This option defaults to <code>True</code> for <code>gbt/gbtr</code> and defaults to <code>False</code> for ensemble <code>gbt/gbtr</code>. When regularization is enabled, <code>regularization_bins</code> defaults to 200.
sample_with_replacement	For each tree, sample the training points using replacement (bootstrap). This option is enabled by default. If turned off, then the <code>sampling_ratio</code> must be provided.
smart_search_seed	When <code>smart_search</code> is enabled, optionally specified a seed value for smart search. If omitted, a time-based seed will be used by default.
table_sampling_seed	Specify a long value for the data sampling random number seed. If omitted, a time-based seed will be used.
testing_objective	Specify one of the following objectives for selecting the test model from a set of tuned models. <code>gbtr.testing_objective.normalized_gini</code> <code>gbtr.testing_objective.mean_absolute_error</code> (default) <code>gbtr.testing_objective.mean_squared_error</code> <code>gbtr.testing_objective.coeff_determination</code>

GLMC Model Configuration

A GLMC model configuration can be specified when training, tuning, and testing models. The following options can be specified in a GLMC configuration.

Table 18: GLMC Model Validation Parameters

Parameter	Description
holdout_ratio	Specify a decimal value for the fraction of the table to be held out for tuning. If this option is specified, a collection of models will be built and evaluated on the held-out tuning set, then the parameters from the best model will be used to train a model on the entire training dataset.
holdout_seed	Optionally specify a long value for the holdout sampling random number seed. This value also serves as the seed for num_folds. If omitted, a time-based seed will be used.
num_folds	Specify an integer for the number of folds for k-fold cross-validation. If holdout_ratio is also specified, then Monte Carlo cross-validation (with randomly drawn holdout sets) is instead performed with the specified number of restarts. If this option is specified, a collection of models will be built and evaluated, then the parameters from the best model will be used to train a model on the entire training dataset.
smart_search	Specify to use an intelligent tuning technique (instead of naive grid search). When this flag is set to True, no other tuning parameters are required; only model validation options are expected. Skytree will automatically search for the best parameter setting. Because Smart Search performs intelligent tuning, this option cannot be used with probability_threshold.
smart_search_iterations	If smart_search is enabled, optionally specify the number of search rounds to try for tuning. This value must be greater than 0 and defaults to 100.

Table 19: Tunable GLMC Parameters specified as a list. When doing a grid search, all possible combinations of values are tried.

Parameter	Description
l1_penalty	Specify the penalty value for L1 (LASSO) regularization. This value must be ≥ 0.0 . Specifying both L1 and L2 values > 0 will result in Elastic Net regularization.
l2_penalty	Specify the penalty value for L2 (Ridge) regularization. This value must be ≥ 0.0 and defaults to 1.0. Specifying both L1 and L2 values > 0 will result in Elastic Net regularization.

Table 20: Non-Tunable GLMC Options

Command	Description
<code>classification_objective</code>	Optionally specify either <code>glm.classification_objective.fscore</code> (default) or <code>glm.classification_objective.accuracy</code> as the objective for classification threshold tuning. This option cannot be used with <code>probability_threshold</code> .
<code>enable_partial_dependencies</code>	Specify whether to export the partial dependencies of the relevant features and/or feature pairs in JSON format. These can be used to generate partial dependence plots. This option is available when: • Training along with (model save OR testing) • Tuning followed by training + (model save OR testing)
<code>enable_pmm1_out</code>	Specify whether to export trained models into PMML (Predictive Model Markup Language) format. This allows the use of an external code for scoring.
<code>epsilon</code>	Specify the numerical accuracy to which to train the GLMC model. This value must be > 0.0 and defaults to 0.001 .
<code>exclude_bias_term</code>	If set, the GLMC formulation will not have a bias term. A bias term is used by default.
<code>k_for_precision</code>	Specify k as a decimal value between 0 and 1 for precision at the top $100k$ -th percentile. This value defaults to 0.1 .
<code>link</code>	Specify the link function to use for the GLMC model. Options include: • <code>glm.link.logit</code> (default): logistic • <code>glm.link.cloglog</code>: complementary log-log
<code>max_cache_memory</code>	Specify the maximum allowed memory, in megabytes, used to cache the rows of the kernel matrix (to reduce recomputation). This value defaults to 500MB.
<code>max_iterations</code>	Specify the maximum allowed number of iterations for the training algorithm. This value must be > 0 . For GLMC training, this corresponds to the number of passes over the data.
<code>probability_threshold</code>	Specify a decimal value for the probability to be used as the threshold for classification. This option cannot be used with <code>classification_objective</code> or <code>testing_objective</code> . Similarly, because <code>probability_threshold</code> cannot be used during tuning, this option cannot be specified with <code>smart_search</code> .
<code>table_sampling_seed</code>	Specify the data sampling random number seed. If omitted, a time-based seed will be used.

Table 20: Non-Tunable GLMC Options (continued)

Command	Description
testing_objective	Specify one of the following objectives for selecting the test model from a set of tuned models. ▪ glm.testing_objective.gini (default) ▪ glm.testing_objective.fscore ▪ glm.testing_objective.accuracy ▪ glm.testing_objective.capture_dev ▪ glm.testing_objective.precision_at_k This option cannot be used with probability_threshold.

GLMR Model Configuration

A GLMR model configuration can be specified when training, tuning, and testing models. The following options can be specified in a GLMR configuration.

Table 21: GLMR Model Validation Parameters

Parameter	Description
holdout_ratio	Specify a decimal value for the fraction of the table to be held out for tuning. If this option is specified, a collection of models will be built and evaluated on the held-out tuning set, then the parameters from the best model will be used to train a model on the entire training dataset.
holdout_seed	Optionally specify a long value for the holdout sampling random number seed. This value also serves as the seed for num_folds. If omitted, a time-based seed will be used.
num_folds	Specify an integer for the number of folds for k-fold cross-validation. If holdout_ratio is also specified, then Monte Carlo cross-validation (with randomly drawn holdout sets) is instead performed with the specified number of restarts. If this option is specified, a collection of models will be built and evaluated, then the parameters from the best model will be used to train a model on the entire training dataset.
smart_search	Specify to use an intelligent tuning technique (instead of naive grid search). When this flag is set to True, no other tuning parameters are required; only model validation options are expected. GLMR will automatically search for the best parameter setting.
smart_search_iterations	If smart_search is enabled, optionally specify the number of search rounds to try for tuning. This value must be greater than 0 and defaults to 100.

Table 22: Tunable GLMR Parameters specified as a list. When doing a grid search, all possible combinations of values are tried.

Parameter	Description
l1_penalty	Specify the penalty value for L1 (LASSO) regularization. This value must be ≥ 0.0 . Specifying both L1 and L2 values > 0 will result in Elastic Net regularization.
l2_penalty	Specify the penalty value for L2 (Ridge) regularization. This value must be ≥ 0.0 and defaults to 1.0. Specifying both L1 and L2 values > 0 will result in Elastic Net regularization.

Table 22: Tunable GLMR Parameters specified as a list. When doing a grid search, all possible combinations of values are tried. (continued)

Parameter	Description
tweedie_exponent	When family is <code>glmr.family.tweedie</code>, specify a value for the exponent. This option has no default value and, therefore, must be explicitly specified when configuring the Tweedie loss function. This value must be > 1.0 and < 2.0. If users desire a value of 1, they should specify the Poisson family and Log link function. Similarly, if users desire an exponent equal to 2.0, they should specify Gamma family along with the Log link function. Note that users can also tune over this option with values > 1.0 and < 2.0. When used for tuning, the output CSV will have an extra column for this value.

Table 23: Non-Tunable GLMR Options

Command	Description
enable_partial_dependencies	Specify whether to export the partial dependencies of the relevant features and/or feature pairs in JSON format. These can be used to generate partial dependence plots. This option is available when: • Training along with (model save OR testing) • Tuning followed by training + (model save OR testing)
enable_pmm1_out	Specify whether to export trained models into PMML (Predictive Model Markup Language) format. This allows the use of an external code for scoring.
epsilon	Specify the numerical accuracy to which to train the GLMR model. This value must be > 0.0 and defaults to 0.001.
exclude_bias_term	If set, the GLMR formulation will not have a bias term. A bias term is used by default. Note: This option cannot be used with the following family-link pairs: Gamma-Inverse, Gamma-Canonical, Poisson-Identity, or Poisson-Sqrt.

Table 23: Non-Tunable GLMR Options (continued)

Command	Description
family	Specify the family function to use for a GLMR model. Options include: • <code>glmr.family.gamma</code> • <code>glmr.family.gaussian</code> • <code>glmr.family.poisson</code> • <code>glmr.family.tweedie</code>
link	Specify the link function to use for the GLMR model. Options include: • <code>glmr.link.canonical</code>. This can be use with any family option. • <code>glmr.link.identity</code>. This can be specified if family is <code>glmr.family.gaussian</code> or <code>glmr.family.poisson</code>. • <code>glmr.link.inverse</code>. This can be specified if family is <code>glmr.family.gamma</code>. • <code>glmr.link.log</code>. This can be specified if family is <code>glmr.family.gamma</code>, <code>glmr.family.poisson</code>, or <code>glmr.family.tweedie</code>. • <code>glmr.link.sqrt</code>. This can be specified if family is <code>glmr.family.poisson</code>.
max_cache_memory	Specify the maximum allowed memory, in megabytes, used to cache the rows of the kernel matrix (to reduce recomputation). This value defaults to 500MB.
max_iterations	Specify the maximum allowed number of iterations for the training algorithm. This value must be > 0. For GLMR training, this corresponds to the number of passes over the data.
table_sampling_seed	Specify the data sampling random number seed. If omitted, a time-based seed will be used.
testing_objective	Specify one of the following objectives for selecting the test model from a set of tuned models. • <code>glmr.testing_objective.normalized_gini</code> • <code>glmr.testing_objective.mean_absolute_error</code> (default) • <code>glmr.testing_objective.mean_squared_error</code> • <code>glmr.testing_objective.coeff_determination</code>

RDF Model Configuration

An RDF model configuration can be specified when training, tuning, and testing models. The following options can be specified in an RDF configuration.

Table 24: RDF Model Validation Parameters

Parameter	Description
holdout_ratio	Specify a decimal value for the fraction of the table to be held out for tuning. If this option is specified, a collection of models will be built and evaluated on the held-out tuning set, then the parameters from the best model will be used to train a model on the entire training dataset.
holdout_seed	Optionally specify a long value for the holdout sampling random number seed. This value also serves as the seed for num_folds. If omitted, a time-based seed will be used.
num_folds	Specify an integer for the number of folds for k-fold cross-validation. If holdout_ratio is also specified, then Monte Carlo cross-validation (with randomly drawn holdout sets) is instead performed with the specified number of restarts. If this option is specified, a collection of models will be built and evaluated, then the parameters from the best model will be used to train a model on the entire training dataset.

Table 25: Tunable RDF Parameters specified as a list. When doing a grid search, all possible combinations of values are tried.

Parameter	Description
imbalance_scale	Specify a list of decimal values for the relative size of all classes with respect to the smallest class. This option is only available if imbalance is enabled.
num_dimensions	Specify a list of integers for the number of dimensions to sample at each node. If omitted, $(\log_2(D) + 1)$ is used, where D is the number of attributes.
num_trees	Required. Specify a list of numbers to indicate the number of trees in the ensemble. This value must be > 0 .
sampling_ratio	Specify a list of doubles to indicate the per-class sampling ratio. You can use a single sampling_ratio that will be used for each class, or specify sampling_ratio values for each class. If a value is not specified and sample_with_replacement is enabled, then full bootstrapping will be used.
smoothing	Specify the probability smoothing parameter. This value must be ≥ 0 . Note that specifying 0 disables this feature.
tree_depth	Specify the depth to which each ensemble is built. If set to 0, trees will be built to their fullest extent.

Table 26: Non-Tunable RDF Parameters

Parameter	Description
<code>cardinality_based_dimension_sampling</code>	Categorical dimensions with larger number of unique values in a given node will have a proportionately higher chance of being selected when sampling dimensions. Continuous features are treated as having 1 unique value. When <code>False</code> , all dimensions have an equal chance of being selected in dimension sampling
<code>categorical_random_split_threshold</code>	If the number of categorical values exceeds this integer value, then random splitting is attempted. Larger values will slow down training time considerably for exhaustive searches of all combinations of $O(2^{q-1})$. For binary classification problems, though, it is $O(q)$ and, therefore, can be set to larger values. This value defaults to 4.
<code>categorical_random_split_tries</code>	Specify an integer for the number of times to try random categorical splits. This value defaults to 10.
<code>categorical_sampling_seed</code>	Specify a long value for the categorical values sampling random number seed. If omitted, a time-based seed will be used.
<code>categorical_selection_method</code>	Specify one of the following methods. <code>rdف.categorical_selection_method.random</code> <code>rdف.categorical_selection_method.random_exact</code> (default) <code>rdف.categorical_selection_method.one_vs_all</code> <code>rdف.categorical_selection_method.one_vs_all_random</code> <code>rdف.categorical_selection_method.one_vs_all_exact</code> <code>rdف.categorical_selection_method.exact</code>
<code>classification_objective</code>	Optionally specify either <code>rdف.classification_objective.fscore</code> (default) or <code>rdف.classification_objective.accuracy</code> as the objective for classification threshold tuning. This option cannot be used with <code>probability_threshold</code> .
<code>dimension_sampling_seed</code>	Specify a long value for the dimension sampling random number seed. If omitted, a time-based seed will be used.
<code>enable_pmm1_out</code>	Specify whether to export trained models into PMML (Predictive Model Markup Language) format. This allows the use of an external code for scoring.
<code>imbalance</code>	When set to <code>True</code> , the API will attempt to improve classification for imbalanced classes. Note that this option cannot be used with <code>sampling_ratio</code> .
<code>impurity</code>	Specify <code>rdف.impurity.gini</code> to use Gini impurity when generating the split. .

Table 26: Non-Tunable RDF Parameters (continued)

Parameter	Description
<code>k_for_precision</code>	Specify k as a decimal value between 0 and 1 for precision at the top 100 k -th percentile. This value defaults to 0.1.
<code>prob_aggregation_method</code>	Specify the method used for aggregation of individual tree predictions. Specify either <code>rdf.prob_aggregation_method.vote</code> or <code>rdf.prob_aggregation_method.average</code> (default).
<code>probability_threshold</code>	Specify a decimal value for the probability to be used as the threshold for classification. This option cannot be used with <code>classification_objective</code> or <code>testing_objective</code> . Similarly, because <code>probability_threshold</code> cannot be used during tuning, this option cannot be specified with <code>smart_search</code> .
<code>sample_with_replacement</code>	For each tree, sample the training points using replacement (bootstrap). This option is set to True by default. If set to False, then the <code>sampling_ratio</code> must be provided.
<code>split_criterion</code>	Specify the method to use (in quotes) as the measure for split criterion. Optional values include the following: • <code>rdf.split_criterion.information_gain</code> • <code>rdf.split_criterion.information_gain_ratio</code> (default)
<code>table_sampling_seed</code>	Specify a long value for the data sampling random number seed. If omitted, a time-based seed will be used.
<code>testing_objective</code>	Specify one of the following objectives for selecting the test model from a set of tuned models. ▪ <code>rdf.testing_objective.gini</code> (default) ▪ <code>rdf.testing_objective.fscore</code> ▪ <code>rdf.testing_objective.accuracy</code> ▪ <code>rdf.testing_objective.capture_dev</code> ▪ <code>rdf.testing_objective.precision_at_k</code> This option cannot be used with <code>probability_threshold</code> .

RDFR Model Configuration

An RDFR model configuration can be specified when training, tuning, and testing models. The following options can be specified in an RDFR configuration.

Table 27: *RDFR Model Validation Parameters*

Parameter	Description
holdout_ratio	Specify a decimal value for the fraction of the table to be held out for tuning. If this option is specified, a collection of models will be built and evaluated on the held-out tuning set, then the parameters from the best model will be used to train a model on the entire training dataset.
holdout_seed	Optionally specify a long value for the holdout sampling random number seed. This value also serves as the seed for num_folds. If omitted, a time-based seed will be used.
num_folds	Specify an integer for the number of folds for k-fold cross-validation. If holdout_ratio is also specified, then Monte Carlo cross-validation (with randomly drawn holdout sets) is instead performed with the specified number of restarts. If this option is specified, a collection of models will be built and evaluated, then the parameters from the best model will be used to train a model on the entire training dataset.

Table 28: *Tunable RDFR Parameters specified as a list. When doing a grid search, all possible combinations of values are tried.*

Parameter	Description
min_node_weight	Specify an array for the minimum bound on the total size in each leaf during tree building.
num_dimensions	Specify a list of integers for the number of dimensions to sample at each node. . By default, RDFR used $(D/3)$ dimensions, where D is the total number of attributes in the dataset.
num_trees	Required. Specify a list of numbers to indicate the number of trees in the ensemble. This value must be > 0 .
sampling_ratio	Specify a list of doubles to indicate the per-class sampling ratio. You can use a single sampling_ratio that will be used for each class, or specify sampling_ratio values for each class. If a value is not specified and sample_with_replacement is enabled, then full bootstrapping will be used.
tree_depth	Specify a list of integers for the depth to which each ensemble is built. If set to 0, trees will be built to the fullest extent.

Table 29: Non-Tunable RDFR Parameters

Parameter	Description
<code>cardinality_based_dimension_sampling</code>	Categorical dimensions with larger number of unique values in a given node will have a proportionately higher chance of being selected when sampling dimensions. Continuous features are treated as having 1 unique value. When set to <code>False</code> , all dimensions have an equal chance of being selected in dimension sampling
<code>dimension_sampling_seed</code>	Specify a long value for the dimension sampling random number seed. If omitted, a time-based seed will be used.
<code>enable_pmm1_out</code>	Specify whether to export trained models into PMML (Predictive Model Markup Language) format. This allows the use of an external code for scoring.
<code>table_sampling_seed</code>	Specify a long value for the data sampling random number seed. If omitted, a time-based seed will be used.
<code>testing_objective</code>	Specify one of the following objectives for selecting the test model from a set of tuned models. ▪ <code>rdfr.testing_objective.normalized_gini</code> ▪ <code>rdfr.testing_objective.mean_absolute_error</code> (default) ▪ <code>rdfr.testing_objective.mean_squared_error</code> ▪ <code>rdfr.testing_objective.coeff_determination</code>

SVM Model Configuration

An SVM model configuration can be specified when training, tuning, and testing classification models that will be used for predictions. The following options can be specified in an SVM configuration.

Table 30: SVM Model Validation Parameters

Command	Description
<code>holdout_ratio</code>	Specify a decimal value for the fraction of the table to be held out for tuning. If this option is specified, a collection of models will be built and evaluated on the held-out tuning set, then the parameters from the best model will be used to train a model on the entire training dataset.
<code>holdout_seed</code>	Optionally specify a long value for the holdout sampling random number seed. This value also serves as the seed for <code>num_folds</code> . If omitted, a time-based seed will be used.
<code>num_folds</code>	Specify an integer for the number of folds for k-fold cross-validation. If <code>holdout_ratio</code> is also specified, then Monte Carlo cross-validation (with randomly drawn holdout sets) is instead performed with the specified number of restarts. If this option is specified, a collection of models will be built and evaluated, then the parameters from the best model will be used to train a model on the entire training dataset.
<code>smart_search</code>	Specify to use an intelligent tuning technique (instead of naive grid search). When this flag is set to <code>True</code>, no other tuning parameters are required; only model validation options are expected. SVM will automatically search for the best parameter setting. Because Smart Search performs intelligent tuning, this option cannot be used with <code>probability_threshold</code>.
<code>smart_search_iterations</code>	If <code>smart_search</code> is enabled, optionally specify the number of search rounds to try for tuning. This value must be greater than 0 and defaults to 100.

Table 31: Tunable SVM Parameters specified as a list

Command	Description
<code>lambda_</code>	The regularization parameter in the SVM formulation. This option is required when training or tuning without smart search. If <code>smart_search</code> is enabled, then this value is optional.
<code>polynomial_degree</code>	Specify the degree of the polynomial kernel. This option is required when training or tuning with a kernel type specified as <code>svm.kernel.polynomial</code> .
<code>polynomial_offset</code>	Specify the offset of the polynomial kernel. This can only be used when the kernel type is specified as <code>svm.kernel.polynomial</code> . Must be ≥ 0 .

Table 31: Tunable SVM Parameters specified as a list (continued)

Command	Description
<code>polynomial_scale</code>	Specify the scale of the polynomial kernel. This can only be used when the kernel type is specified as <code>svm.kernel.polynomial</code> . Must be ≥ 0 .
<code>rbf_bandwidth</code>	Specify the a floating point value as the bandwidth of the RBF kernel. Must be ≥ 0 . This option is required when training or tuning with a kernel type specified as <code>svm.kernel.rbf</code> .

Table 32: SVM Non-Tunable Parameters

Command	Description
<code>classification_objective</code>	Optionally specify either <code>svm.classification_objective.fscore</code> (default) or <code>svm.classification_objective.accuracy</code> as the objective for classification threshold tuning. This option cannot be used with <code>probability_threshold</code> .
<code>enable_partial_dependencies</code>	Specify whether to export the partial dependencies of the relevant features and/or feature pairs in JSON format. These can be used to generate partial dependence plots. This option is available when: • Training along with (model save OR testing) • Tuning followed by training + (model save OR testing)
<code>enable_pmm1_out</code>	Specify whether to export trained models into PMML (Predictive Model Markup Language) format. This allows the use of an external code for scoring.
<code>epsilon</code>	Specify the numerical accuracy to which to train the SVM. This defaults to $1e-3$.
<code>exclude_bias_term</code>	If enabled, the SVM formulation will not have a bias term. A bias term is used by default.
<code>k_for_precision</code>	Specify k as a decimal value between 0 and 1 for precision at the top 100k-th percentile. This value defaults to 0.1.
<code>kernel</code>	Specify the kernel function to be used for the SVM. Specify <code>svm.kernel.linear</code> (default), <code>svm.kernel.rbf</code> (Gaussian), or <code>svm.kernel.polynomial</code> . Note: An SVM smart search run is based on a specified kernel type. If <code>kernel</code> is not specified, smart search will run using type <code>svm.kernel.linear</code> . If a different kernel function is desired in smart search, then it must be explicitly specified.
<code>max_cache_memory</code>	Specify the maximum allowed memory, in megabytes, used to cache the rows of the kernel matrix (to reduce recomputation). This value defaults to 500MB for nonlinear SVM.

Table 32: SVM Non-Tunable Parameters (continued)

Command	Description
<code>max_iterations</code>	Specify the maximum allowed number of iterations for the training algorithm. This value must be > 0. For linear SVM training, this corresponds to the number of passes over the data.
<code>probability_threshold</code>	Specify a decimal value for the probability to be used as the threshold for classification. This option cannot be used with <code>classification_objective</code> or <code>testing_objective</code> . Similarly, because <code>probability_threshold</code> cannot be used during tuning, this option cannot be specified with <code>smart_search</code> .
<code>smart_search_seed</code>	When <code>smart_search</code> is enabled, optionally specified a seed value for smart search. If omitted, a time-based seed will be used by default.
<code>table_sampling_seed</code>	Specify a long value for the data sampling random number seed. If omitted, a time-based seed will be used.
<code>testing_objective</code>	Specify one of the following objectives for selecting the test model from a set of tuned models. ▪ <code>svm.testing_objective.gini</code> (default) ▪ <code>svm.testing_objective.fscore</code> ▪ <code>svm.testing_objective.accuracy</code> ▪ <code>svm.testing_objective.capture_dev</code> ▪ <code>svm.testing_objective.precision_at_k</code> This option cannot be used with <code>probability_threshold</code>.

Model Examples

This section includes usage examples for creating models using the available functions. Refer to the following sections for more information:

- [AutoModel Examples](#) (page 142)
- [GBT Model Examples](#) (page 142)
- [GBTR Model Examples](#) (page 143)
- [GLM Model Examples](#) (page 144)
- [RDF Model Examples](#) (page 146)
- [RDFR Model Examples](#) (page 147)
- [SVM Model Examples](#) (page 147)

AutoModel Examples

This section includes the following SDK usage example for creating AutoModel models.

- [A Simple Regression Example](#) (page 142)

A Simple Regression Example

The following example shows how to build a simple model. The AutoModel module automatically recognizes this as a regression problem because the `objective_column` is continuous rather than categorical.

Create configuration

```
automodel = automodel.Config()  
automodel.num_folds = 10
```

Specify the configuration when creating the model

```
automodelreg = skytree.prediction.learn(  
    incomeid, "capital-gain", automodel, name="AutoModelRegression"  
) .ready()
```

GBT Model Examples

This section includes the following advanced SDK usage examples for creating GBT models.

- [Tuning the Number of Trees with a Holdout Set](#) (page 142)
- [Tuning with Smart Search](#) (page 143)

Tuning the Number of Trees with a Holdout Set

This example specifies to hold out 20% of the training data for use in evaluating the model. The default behavior is to score the model using the tuning set after every 10% of iterations (10 in this case). Thus, the model will build 10 trees

and score on the holdout data. Then it will go on to build 20 trees and score on the holdout data and so on, reporting accuracy scores each step of the way.

Create configuration

```
GbtHoldout = gbt.Config()  
GbtHoldout.num_trees = 10  
GbtHoldout.holdout_ratio = .2
```

Specify the configuration when creating the model

```
GbtModel = skytree.prediction.learn(  
    incomeid, "sex", GbtHoldout, name="GbtModel"  
) .ready()
```

Tuning with Smart Search

The following example shows how to tune using GBT smart search with a configuration. The configuration includes the smart search model validation options for a holdout ratio and holdout seed. The number of smart search runs is specified as 20. This means that GBT will run 20 times and use different parameter settings each time.

Create configuration

```
GbtSmartSearch = gbt.Config()  
GbtSmartSearch.holdout_ratio = .3  
GbtSmartSearch.holdout_seed = 1  
GbtSmartSearch.smart_search = True  
GbtSmartSearch.smart_search_iterations = 20
```

Specify the configuration when creating the model

```
GbtSmartSearchModel = skytree.prediction.learn(  
    incomeid, "sex", GbtSmartSearch, name="GbtSmartSearch"  
) .ready()
```

GBTR Model Examples

This section includes the following advanced SDK usage examples for creating GBTR models.

- [Tuning with a Loss Function](#) (page 143)
- [Tuning the Number of Trees using K-Fold Cross Validation](#) (page 144)

Tuning with a Loss Function

In GBTR, the default loss function method performs least absolute deviation (Lad). Certain cases can exist, however, in which this loss function can lead to poor results. The `loss_function` option allows you to specify a different method to use when performing regression. The example below specifies to use the least squares (Ls) loss function.

Create configuration

```
LsLossFunction = gbtr.Config()  
LsLossFunction.num_trees = [5,15,25]  
LsLossFunction.holdout_ratio = .2
```

```
LsLossFunction.loss_function = gbtr.loss_function.ls
```

Specify the configuration when creating the model

```
GbtrModel = skytree.prediction.learn(  
    incomeid, "education-num", LsLossFunction, name="GbtrLossFunction"  
).ready()
```

Tuning the Number of Trees using K-Fold Cross Validation

Tuning can also be done using K-fold cross validation. This is better than using a single hold out set, which is randomly generated. The downside is that it can be K times slower. This example uses the `num_folds` option to return the best average results over all folds. This method can be used to fine tune parameters after honing in on approximate parameters using a holdout set.

Note: If both `num_folds` and `holdout_ratio` are provided in a single command, then the algorithm will not do K-fold cross validation but instead repeat `num_folds` times with a new holdout tuning set and return the parameters with the best average results over these `num_folds` runs.

Create configuration

```
GbtrKFold = gbtr.Config()  
GbtrKFold.num_trees = [5,15,25]  
GbtrKFold.num_folds = 5
```

Specify the configuration when creating the model

```
GbtrKFold = skytree.prediction.learn(  
    incomeid, "education-num", GbtrKFold, name="GbtrKFold"  
).ready()
```

GLM Model Examples

This section includes the following advanced SDK usage examples for creating GLMR and GLMC models.

- [Training a Gaussian-Identity Model](#) (page 144)
- [Excluding the Bias Term](#) (page 145)
- [Tuning an Unbiased Poisson-Log Model](#) (page 145)

Training a Gaussian-Identity Model

The following example trains a model using the gaussian-identity Family/Link Function pair.

Create configuration

```
glmr1 = glmr.Config()  
glmr1.family = glmr.family.gaussian  
glmr1.link = glmr.link.identity  
glmr1.l1_penalty = 0.01  
glmr1.l2_penalty = 0  
glmr1.epsilon = 0.001
```

```
glmr1.max_iterations = 1000
```

Specify the configuration when creating the model

```
GlrmModel = skytree.prediction.learn(  
    regression_dataset, "data0", glmr1, name="GLMR G-I Model"  
) .ready()
```

Excluding the Bias Term

By default, a bias term is included in the computed model. The bias term adjusts all predictions up or down by a constant amount, i.e. it is the predicted value when all inputs are exactly 0. It can be excluded (forced equal 0) with "excludeBiasTerm": true.

In GLM, the inverse of the link function is applied to obtain the final predictions (targets in GLMR and probabilities in GLMC). Excluding the bias term can negatively impact measured model quality, but it is appropriate when a given linear model should definitely predict 0 when all inputs are 0 (before the application of the inverse of the link function to obtain the final predictions). You may also want to exclude the bias/intercept term to choose a simpler model when appropriate and move away from overfitting.

Note: This option cannot be used with the following family/link combinations: poisson/identity, poisson/sqrt, and gamma/inverse.

The following example shows how to exclude the Bias term in a GLM model. This example trains a classification (GLMC) model.

Create configuration

```
glmc = glmc.Config()  
glmc.link = glm.link.cloglog  
glmc.l1_penalty = 0.01  
glmc.l2_penalty = 0  
glmc.epsilon = 0.001  
glmc.max_iterations = 1000  
glmc.exclude_bias_term = on
```

Specify the configuration when creating the model

```
GlmcModel = skytree.prediction.learn(  
    classification_dataset, "column_5", glmc, name="GLMC Exclude Bias Term"  
) .ready()
```

Tuning an Unbiased Poisson-Log Model

The following example tunes an unbiased GLMR model using the poisson-log Family/Link Function pair.

Create configuration

```
glmr2 = glmr.Config()  
glmr2.num_folds = 5  
glmr2.family = glmr.family.poisson  
glmr2.link = glmr.link.log  
glmr2.exclude_bias_term = on
```

```
glmr2.l1_penalty = 0,0.001,0.01,0.1
glmr2.l2_penalty = 0,0.001,0.01,0.1
```

Specify the configuration when creating the model

```
GLmrPLModel = skytree.prediction.learn(
    regression_dataset, "data0", glmr2, name="GLMR P-L Model"
).ready()
```

RDF Model Examples

This section includes the following advanced SDK usage examples for creating RDF models.

- [Tuning the Number of Dimensions](#) (page 146)
- [Tuning for Skewed Data Automatically](#) (page 146)

Tuning the Number of Dimensions

This example shows how to tune the number of randomly selected dimensions considered in splitting each tree node. By default, RDF uses $\log_2(D + 1)$ dimensions, where D is the total number of attributes in the dataset.

Create configuration

```
RdfDimensions = rdf.Config()
RdfDimensions.num_trees = 100
RdfDimensions.holdout_ratio = .2
RdfDimensions.num_dimensions = [1,2,4]
```

Specify the configuration when creating the model

```
RdfDimModel = skytree.prediction.learn(
    incomeid, "sex", RdfDimensions, name="RdfDimModel"
).ready()
```

Tuning for Skewed Data Automatically

Often, the class distribution in the data is skewed, with far fewer points in one of the classes. Skytree provides a `sampling_ratio` option for up-sampling or down-sampling each class separately, in order to build a model that better accounts for the smaller class's distribution. Alternatively, the `imbalance` option can be used to automatically handle skewed data.

Create configuration

```
RdfImbalance = rdf.Config()
RdfImbalance.imbalance = True
RdfImbalance.num_trees = 10
```

Specify the configuration when creating the model

```
RdfImbalance = skytree.prediction.learn(
    incomeid, "sex", RdfImbalance, name="RdfImbalanceModel"
).ready()
```


RDFR Model Examples

This section includes the following advanced SDK usage examples for creating RDFR models.

- [Tuning for the Sampling Ratio](#) (page 147)
- [Tuning the Minimum Node Size](#) (page 147)

Tuning for the Sampling Ratio

The `sampling_ratio` option can be used to change the fraction of the data that is used in each tree of a random forest.

Create configuration

```
RdfrSampling = rdfr.Config()
RdfrSampling.num_trees = [100]
RdfrSampling.holdout_ratio = 0.2
RdfrSampling.sampling_ratio = [1.0, 2.3]
```

Specify the configuration when creating the model

```
RdfrSampRatio = skytree.prediction.learn(
    incomeid, "education-num", configuration, name="RdfrSampModel"
).ready()
```

Tuning the Minimum Node Size

Use the `min_node_weight` option to specify the minimum bound on the total size in each leaf during tree building.

Create configuration

```
RdfrNodewght = rdfr.Config()
RdfrNodewght.num_trees = [100]
RdfrNodewght.holdout_ratio = 0.4
RdfrNodewght.min_node_weight = [10,100]
```

Specify the configuration when creating the model

```
RdfrNodewght = skytree.prediction.learn(
    incomeid, "education-num", RdfrNodewght, name="RdfrNodeweight"
).ready()
```

SVM Model Examples

This section includes the following SDK usage examples for creating SVM models.

- [Tuning a Non-Linear SVM](#) (page 148)
- [Tuning for Best Accuracy](#) (page 148)

Tuning a Non-Linear SVM

This example tunes over regularization values 0.01, 0.1 and RBF (radial-basis function) kernels with bandwidths 0.1, 1.0, and 10.0 (for a total of 6 parameters settings). Note that unlike other examples that use the income.data dataset, the SVM models will use an SvmData dataset. This is because SVM cannot be used with missing values.

Create configuration

```
SvmNonLinear= svm.Config()  
SvmNonLinear.num_folds = 10  
SvmNonLinear.kernel = svm.kernel.rbf  
SvmNonLinear.lambda_ = [0.1,0.1]  
SvmNonLinear.rbf_bandwidth = [0.1,1.0,10.0]
```

Specify the configuration when creating the model

```
SvmModel = skytree.prediction.learn(  
    SvmData, "gender", SvmNonLinear, name="SvmModel"  
) .ready()
```

Tuning for Best Accuracy

During tuning, Skytree tunes for the best parameters and uses the model giving the best Gini by default. If you want to pick the model with the best accuracy rather than the best Gini, you can specify this with the `testing_objective` option.

Create configuration

```
SvmAccuracy= svm.Config()  
SvmAccuracy.num_folds = 10  
SvmAccuracy.lambda_ = [0.1,0.1,0.5]  
SvmAccuracy.testing_objective = svm.testing_objective.accuracy
```

Specify the configuration when creating the model

```
SvmModel = skytree.prediction.learn(  
    SvmData, "gender", SvmNonLinear, name="SvmAccuracyModel"  
) .ready()
```

Appendix B Custom Script Objects

This section describes the objects that Skytree provides for the user to use in custom scripts. Consult the Spark documentation for further details on behavior and functions of the Spark objects and methods.

- [sc](#) (page 149)
- [Input](#) (page 150)
- [Output](#) (page 151)
- [Column](#) (page 151)
- [RDD](#) (page 151)
- [Double RDD](#) (page 154)
- [Pair RDD](#) (page 155)

SC

A restricted version of SparkContext.

Methods:

Method signatures are provided for the underlying Scala functions. In a Skytree custom script, these would be accessed through Python.

Create an Accumulable shared variable of the given type, to which tasks can "add" values with add.

```
<T,R> Accumulable<T,R> accumulable(T initialValue, AccumulableParam<T,R> param)
```

Create an Accumulator variable of a given type, which tasks can "add" values to using the add method.

```
<T> Accumulator<T> accumulator(T initialValue, AccumulatorParam<T> accumulatorParam)
```

Create an Accumulator integer variable, which tasks can "add" values to using the add method.

```
Accumulator<Integer> accumulator(Integer initialValue)
```

Create an Accumulator double variable, which tasks can "add" values to using the add method.

```
Accumulator<Double> accumulator(double initialValue, String name)
```

Application name

```
String appName()
```

Broadcast a read-only variable to the cluster, returning a Broadcast object for reading it in distributed functions.

```
<T> Broadcast<T> broadcast(T value)
```

Default min number of partitions for Hadoop RDDs when not given by user

```
Integer defaultMinPartitions()
```

Default level of parallelism to use when not given by user (e.g. parallelize).

```
Integer defaultParallelism()
```

Create an Accumulator double variable, which tasks can "add" values to using the add method.

```
Accumulator<Double> doubleAccumulator(double initialValue)
```

Get a local property set in this thread, or null if it is missing.

```
String getLocalProperty(String key)
```

Get Spark's home location from either a value set through the constructor, or the spark.home Java property, or the SPARK_HOME environment variable (in that order of preference).

```
String getSparkHome()
```

Create an Accumulator integer variable, which tasks can "add" values to using the add method.

```
Accumulator<Integer> intAccumulator(int initialValue)
```

Is Spark running in Local mode

```
Boolean isLocal()
```

Distribute a local Scala collection to form an RDD.

```
<T> JavaRDD<T> parallelize(java.util.List<T> list)
```

Distribute a local Scala collection to form an RDD.

```
<T> JavaRDD<T> parallelize(java.util.List<T> list, int numSlices)
```

Distribute a local Scala collection to form an RDD.

```
JavaDoubleRDD parallelizeDoubles(java.util.List<Double> list)
```

Distribute a local Scala collection to form an RDD.

```
JavaDoubleRDD parallelizeDoubles(java.util.List<Double> list, int numSlices)
```

Distribute a local Scala collection to form an RDD.

```
<K,V> JavaPairRDD<K,V> parallelizePairs(java.util.List<scala.Tuple2<K,V>> list)
```

Distribute a local Scala collection to form an RDD.

```
<K,V> JavaPairRDD<K,V> parallelizePairs(java.util.List<scala.Tuple2<K,V>> list, int numSlices)
```

Context start time.

```
Long startTime()
```

Input

Description:

Array of input RDDs. An input RDD is of type `SkytreeJavaRDD<String[]>`.

Methods:

Method signatures are provided for the underlying Scala functions. In a Skytree custom script, these would be accessed through Python.

Gets a list of column names for the RDD

```
String[] getColumnNames()
```

Gets a list of Column objects

```
List<Column> getColumns()
```

Gets the Column object for a given column name

```
Column getColumn(String columnName)
```

Output

Description:

RDD or Array of RDDs sent back to the system

Methods:

Method signatures are provided for the underlying Scala functions. In a Skytree custom script, these would be accessed through Python

Sets the column names for the output RDD

```
void setColumnNames(String[] columnNames)
```

Sets the type of the given column to Text

```
void setColumnTypeText(String columnName)
```

Sets the type of the given column to Integer

```
void setColumnTypeInteger(String columnName)
```

Sets the type of the given column to Double

```
void setColumnTypeFloat(String columnName)
```

Sets the given column's categorical flag

```
void setColumnCategorical(String columnName, boolean isCategorical)
```

Sets the given column's blob flag

```
void setColumnTypeBlob(String columnName)
```

Column

Methods:

Method signatures are provided for the underlying Scala functions. In a Skytree custom script, these would be accessed through Python.

Gets the name of the column

```
string getName()
```

Gets the 1-based index of the column

```
long getIndex()
```

Returns True if the column is marked as categorical

```
boolean isCategorical()
```

Returns the type of the column. One of Text, Integer, Float

```
type getType()
```

RDD

Methods

Method signatures are provided for the Python calls available to the user.

Return a new RDD containing only the elements that satisfy a predicate function `f`, which takes as input an RDD row.

```
filter(f)
```

Aggregate the elements of each partition, and then the results for all the partitions, using given combine functions and a neutral "zero value".

```
aggregate(zeroValue, seqOp, combOp)
```

Return a new RDD by first applying a function to all elements of this RDD, and then flattening the results.

```
flatMap(f)
```

Return a new Pair-RDD by first applying a function to all elements of this RDD, and then flattening the results.

```
flatMapToPair(f)
```

Aggregate the elements of each partition, and then the results for all the partitions, using a given associative function and a neutral "zero value".

```
fold(zeroValue, f)
```

Applies a function `f` to all elements of this RDD.

```
foreach(f)
```

Applies a function `f` to each partition of this RDD.

```
foreachPartition(f)
```

Return a Pair-RDD of grouped elements.

```
groupBy(f)
```

Return a Pair-RDD of grouped elements with the given number of partitions.

```
groupBy(f, numPartitions)
```

Creates a Pair-RDD of tuples of the elements in this RDD by applying `f`.

```
keyBy(f)
```

Return a new RDD by applying a function to all elements of this RDD.

```
map(f)
```

Return a new RDD by applying a function to each partition of this RDD.

```
mapPartitions(f)
```

Return a new RDD by applying a function to each partition of this RDD. Set `preservesPartitioning` to `True` to keep the same partitions.

```
mapPartitions(f, preservesPartitioning)
```

Return a new Pair-RDD by applying a function to each partition of this RDD.

```
mapPartitionsToPair(f)
```

Return a new RDD by applying a function to each partition of this RDD, while tracking the index of the original partition. Set `preservesPartitioning` to `True` to preserve the original partitioning.

```
mapPartitionsWithIndex(f, preservesPartitioning)
```

Return a new Pair-RDD by applying a function to all elements of this RDD.

```
mapToPair(f)
```

Return a new RDD that is reduced into `numPartitions` partitions.

```
coalesce(numPartitions)
```

Return an array that contains all of the elements in this RDD.

```
collect()
```

Return the number of elements in the RDD.

```
count()
```

Return an array that contains all of the elements in specific partitions of this RDD.

```
collectPartitions(partitionIds)
```

Return the count of each unique value in this RDD as a map of (value, count) pairs.

```
countByValue()
```

Return a new RDD containing the distinct elements in this RDD.

```
distinct()
```

Return the first element in this RDD.

```
first()
```

Return the number of partitions in this RDD.

```
getNumPartitions()
```

Return an RDD created by coalescing all elements within each partition into an array.

```
glom()
```

Return the intersection of this RDD and another one.

```
intersection(rdd)
```

Returns the maximum element from this RDD as defined by the specified comparator f.

```
max(f)
```

Returns the minimum element from this RDD as defined by the specified comparator f.

```
min(f)
```

Reduces the elements of this RDD using the specified commutative and associative binary operator f.

```
reduce(f)
```

Return a new RDD that has exactly numPartitions partitions.

```
repartition(numPartitions)
```

Randomly selects a fraction of the items of a RDD and returns them in a new RDD.

```
sample(withReplacement, fraction)
```

Randomly selects a fraction of the items of a RDD and returns them in a new RDD.

```
sample(withReplacement, fraction, seed)
```

Set of partitions in this RDD.

```
splits()
```

Take the first num elements of the RDD.

```
take(num)
```

Return an RDD with the elements from this that are not in other.

```
subtract(rdd)
```

Return an RDD with the elements from this that are not in other.

```
subtract(rdd, numPartitions)
```

Get the N elements from a RDD ordered in ascending order.

```
takeOrdered(num)
```

Return a fixed-size sampled subset of this RDD (currently requires numpy).

```
takeSample(withReplacement, num)
```

Return a fixed-size sampled subset of this RDD.

```
takeSample(withReplacement, num, seed)
```

Returns the top num elements from this RDD using the natural ordering for T.

```
top(num)
```

Return the union of this RDD and another one.

```
union(rdd)
```

Zips this RDD with another one, returning key-value pairs (Pair-RDD) with the first element in each RDD, second element in each RDD, etc.

```
zip(rdd)
```

Zip this RDD's partitions with one (or more) RDD(s) and return a new RDD by applying a function to the zipped partitions.

```
zipPartitions(rdd, f)
```

Zips this RDD with its element indices. (Requires a pass through the data to determine partition lengths)

```
zipWithIndex()
```

Zips this RDD with generated unique Long ids.

```
zipWithUniqueId()
```

Return a new Double-RDD by first applying a function to all elements of this RDD, and then flattening the results.

```
flatMapToDouble(f)
```

Return a new Double-RDD by applying a function to each partition of this RDD.

```
mapPartitionsToDouble(f)
```

Return a new Double-RDD by applying a function to each partition of this RDD.

```
mapPartitionsToDouble(f, preservesPartitioning)
```

Return a new Double-RDD by applying a function to all elements of this RDD.

```
mapToDouble(f)
```

Double RDD

Methods

Compute a histogram using the provided buckets.

```
histogram(buckets)
```

Compute the mean of this RDD's elements.

```
mean()
```

Return a StatCounter object that captures the mean, variance and count of the RDD's elements in one operation.

```
stats()
```

Compute the variance of this RDD's elements.

```
variance()
```

Add up the elements in this RDD.

```
sum()
```

Compute the standard deviation of this RDD's elements.

```
stdev()
```

Compute the sample standard deviation of this RDD's elements (which corrects for bias in estimating the standard deviation by dividing by N-1 instead of N).

```
sampleStdev()
```

Compute the sample variance of this RDD's elements (which corrects for bias in estimating the standard variance by dividing by N-1 instead of N).

```
samplevariance()
```


Return the count of each unique value in this RDD as a map of (value, count) pairs. The final combine step happens locally on the master, equivalent to running a single reduce task.

```
countByValue()
```

Return the union of this RDD with another.

```
union(rdd)
```

Applies a function f to all elements of this RDD.

```
foreach(f)
```

Return an array that contains all of the elements in this RDD.

```
collect()
```

Return a new Double-RDD containing only the elements that satisfy a predicate.

```
filter(f)
```

Return the first element in this RDD.

```
first()
```

Return the intersection of this RDD and another one.

```
intersection(rdd)
```

Return a new Double-RDD that has exactly numPartitions partitions.

```
repartition(numPartitions)
```

Return an RDD with the elements from this that are not in other.

```
subtract(rdd, numPartitions)
```

Return a new RDD by applying a function to all elements of this RDD.

```
map(f)
```

Pair RDD

Methods

Aggregate the elements of each partition, and then the results for all the partitions, using given combine functions and a neutral "zero value".

```
aggregate(zeroValue, seqOp, combOp)
```

Return the number of elements in the RDD.

```
count()
```

Return an array that contains all of the elements in specific partitions of this RDD.

```
collectPartitions(partitionIds)
```

Generic function to combine the elements for each key using a custom set of aggregation functions.

```
combineByKey(createCombiner, mergeValue, mergeCombiners)
```

Generic function to combine the elements for each key using a custom set of aggregation functions.

```
combineByKey(createCombiner, mergeValue, mergeCombiners, numPartitions)
```

Return the count of each unique value in this RDD as a map of (value, count) pairs.

```
countByValue()
```

Return a new RDD containing the distinct elements in this RDD.

```
distinct()
```

Return the first element in this RDD.

```
first()
```

Return the number of partitions in this RDD.

```
getNumPartitions()
```

Return an RDD created by coalescing all elements within each partition into an array.

```
glom()
```

Return the intersection of this RDD and another one.

```
intersection(rdd)
```

Sort the RDD by key, so that each partition contains a sorted range of the elements.

```
sortByKey()
```

Sort the RDD by key, so that each partition contains a sorted range of the elements. Set ascending to True to get an ascending sort.

```
sortByKey(ascending)
```

Return an RDD with the values of each tuple.

```
values()
```

Return an RDD with the keys of each tuple.

```
keys()
```

Return a new RDD by applying a function to all elements of this RDD.

```
map(f)
```

Return a new RDD that is reduced into numPartitions partitions.

```
coalesce(numPartitions)
```

Return a new RDD that is reduced into numPartitions partitions.

```
coalesce(numPartitions, shuffle)
```

For each key k in this RDD or the other RDD, return a resulting Pair-RDD that contains a tuple with the list of values for that key in this as well as other.

```
cogroup(other)
```

For each key k in this or other1 or other2, return a resulting Pair-RDD that contains a tuple with the list of values for that key in this, other1 and other2.

```
cogroup(other1, other2)
```

Return an array that contains all of the elements in this RDD.

```
collect()
```

Return the key-value pairs in this RDD to the master as a Map.

```
collectAsMap()
```

Count the number of elements for each key, and return the result to the master as a Map.

```
countByKey()
```

Return a new RDD containing the distinct elements in this RDD.

```
distinct()
```

Merge the values for each key using an associative function "func" and a neutral "zeroValue" which may be added to the result an arbitrary number of times, and must not change the result (e.g., 0 for addition, or 1 for multiplication.).

```
foldByKey(zeroValue, f)
```

Merge the values for each key using an associative function "func" and a neutral "zeroValue" which may be added to the result an arbitrary number of times, and must not change the result (e.g., 0 for addition, or 1 for multiplication.).

```
foldByKey(zeroValue, f, numPartitions)
```

Applies a function f to all elements of this RDD.

```
foreach(f)
```

Return a Pair-RDD of grouped elements.

```
groupBy(f)
```

Alias for cogroup but with support for multiple RDDs.

```
groupWith(other)
```

Alias for cogroup but with support for multiple RDDs.

```
groupWith(other1, other2)
```

Group the values for each key in the RDD into a single sequence.

```
groupByKey(numPartitions)
```

Creates a Pair-RDD of tuples of the elements in this RDD by applying f.

```
keyBy(f)
```

Return an RDD containing all pairs of elements with matching keys in this and other.

```
join(rdd)
```

Return an RDD containing all pairs of elements with matching keys in this and other.

```
join(red, numPartitions)
```

Perform a left outer join of this and other.

```
leftOuterJoin(rdd)
```

Perform a left outer join of this and other.

```
leftOuterJoin(rdd, numPartitions)
```

Perform a right outer join of this and other.

```
rightOuterJoin(rdd)
```

Perform a right outer join of this and other.

```
rightOuterJoin(rdd, numPartitions)
```

Return the list of values in the RDD for key key.

```
lookup(key)
```

Merge the values for each key using an associative reduce function.

```
reduceByKey(f)
```

Return a new Pair-RDD that has exactly numPartitions partitions.

```
repartition(numPartitions)
```

Randomly selects a fraction of the items of a RDD and returns them in a new RDD.

```
sample(withReplacement, fraction)
```

Randomly selects a fraction of the items of a RDD and returns them in a new RDD.

```
sample(withReplacement, fraction, seed)
```

Set of partitions in this RDD.

```
splits()
```

Take the first num elements of the RDD.

```
take(num)
```

Return an RDD with the elements from this that are not in other.

```
subtract(rdd)
```

Return an RDD with the elements from this that are not in other.

```
subtract(rdd, numPartitions)
```

Return each (key, value) pair in this rdd that has no pair with matching key in other.

```
subtractByKey(other)
```

Return each (key, value) pair in this rdd that has no pair with matching key in other.

```
subtractByKey(other, numPartitions)
```

Get the N elements from a RDD with the ordering specified using comparator function f.

```
takeOrdered(num, f)
```

Return a fixed-size sampled subset of this RDD (currently requires numpy).

```
takeSample(withReplacement, num)
```

Return a fixed-size sampled subset of this RDD.

```
takeSample(withReplacement, num, seed)
```

Returns the top num elements from this RDD using the natural ordering for T.

```
top(num)
```

Return the union of this RDD and another one.

```
union(rdd)
```

Zips this RDD with another one, returning key-value pairs (Pair-RDD) with the first element in each RDD, second element in each RDD, etc.

```
zip(rdd)
```


Index

A

- add_column 49
- add_custom_transform 20
- add_unique_id_column 50
- apply_custom_transform 75
- apply_transform_steps 73
- Assumptions 1
- authenticate 10
- AutoModel
 - Examples 142
 - Model Configuration 118
 - Models 114

C

- clamp_values 51
- Classification Score Configuration
 - Properties 81
- commit 44
- Configuration
 - AutoModel 118
 - GBT Models 119
 - GBTR Models 124
 - GLMC Models 128
 - GLMR Models 131
 - RDF Models 134
 - RDFR Models 137
 - SVM Models 139
- create_dataset 23
- create_labels_dataset 100
- create_probabilities_dataset 99
- create_project 11
- create_targets_dataset 102
- Custom Script Variables 149
- Custom Transform
 - add_custom_transform 20
- Custom Transforms
 - Properties 9

D

- Datasets
 - apply_custom_transform 75
 - apply_transform_steps 73
 - commit 44
 - create_dataset 23
 - create_labels_dataset 100
 - create_probabilities_dataset 99

- create_targets_dataset 102
- get_log 42
- Properties 8
- sample 45
- summary 46
- terminate 43

- delete_dataset 37
- delete_model 38
- delete_project 15
- delete_result 39
- delete_source 19

E

- extract_text_features 53

F

- filter 54
- findall() 96
- findById() 97

G

- GBT
 - Examples 142
 - Model Configuration 119
 - Models 114

GBTR

- Examples 143,147
- Model Configuration 124
- Models 114

- Generalized Linear Model (see also GLM) 114

- get_dataset 30
- get_dataset_dependents 30
- get_labels 110
- get_log 42,87,108
- get_model 31
- get_model_dependents 32
- get_pdp_variable_names 88
- get_plot 35
- get_plot_dependents 36
- get_pmml_gzip 90
- get_probabilities 109
- get_result 33
- get_result_dependents 34
- get_source_dependents 18
- get_targets 111
- GLM 114
 - Examples 144

- GLMC
 - Model Configuration 128
- GLMR
 - Model Configuration 131
- H
 - horizontalize 55
- I
 - Import 3
 - ingest_text 57
 - ingest_url 58
 - inner_join 59
 - Installation 3
 - Instance Methods
 - Datasets 6
 - Models 77
 - Projects 5
 - Test Results 77
- L
 - left_outer_join 60
 - list_datasets 26
 - list_models 27
 - list_plots 29
 - list_results 28
- M
 - Model Configuration 117
 - Model Examples 142
 - AutoModel 142
 - GBT 142
 - GBTR 143
 - GLM 144
 - RDF 146
 - RDFR 147
 - SVM 147
 - Models 113
 - AutoModel 114
 - GBT 114
 - GBTR 114
 - get_log 87
 - get_pdp_variable_names 88
 - get_pmml_gzip 90
 - GLM 114
 - Instance Methods 77
 - Properties 78
 - RDF 115
 - RDFR 115
 - skytree.prediction.learn 82
 - summary 85
 - SVM 116
 - terminate 84
 - visualize_pdp 92

- Modules
 - skytree 5
 - skytree.prediction 77
- N
 - normalize 61
- P
 - PlotRegistry() 95
 - Plots
 - terminate 98
 - Projects
 - create_project 11
 - delete_dataset 37
 - delete_model 38-39
 - delete_project 15
 - get_dataset 30-31
 - get_dataset_dependents 30
 - get_model_dependents 32
 - get_plot 35
 - get_plot_dependents 36
 - get_result 33
 - get_result_dependents 34
 - Instance Methods 5
 - list_datasets 26
 - list_models 27
 - list_plots 29
 - list_results 28
 - Properties 7
 - rename
 - Projects 41
 - skytree.get_project 13
 - skytree.get_project_dependents 14
 - skytree.list_projects 12
- Properties
 - Classification Score Configuration 81
 - Custom Transforms 9
 - Datasets 8
 - Models 78
 - Projects 7
 - Test Results 79
- Python
 - Importing the Module 3
 - Installation 3
 - Requirements 3
- Python SDK Assumptions 1
- R
 - RDF
 - Examples 146
 - Model Configuration 134
 - Models 115
 - RDFR
 - Model Configuration 137

- Models 115
- ready 7, 78
- remove_columns 63
- rename 41
- right_outer_join 64
- S
- sample 45
- set_as_categorical_number 65
- set_as_categorical_text 66
- set_as_id 68
- skytree Module 5
- skytree.get_custom_transform 22
- skytree.get_project 13
- skytree.get_project_dependents 14
- skytree.get_source 17
- skytree.list_custom_transforms 21
- skytree.list_projects 12
- skytree.list_sources 16
- skytree.prediction Module 77
- skytree.prediction.learn 82
- Sources
 - delete_source 19
 - get_source_dependents 18
 - skytree.get_project 17
 - skytree.list_sources 16
- split 69
- summary
 - Datasets 46
 - Models 85
 - Test Results 103
- SVM 116
 - Examples 147
 - Model Configuration 139
 - Models 116
- T
- terminate 43, 84, 98, 104
- test 93
- Test Results
 - get_labels 110
 - get_log 108
 - get_probabilities 109
 - get_targets 111
 - Instance Methods 77
 - Properties 79
 - summary 103
 - terminate 104
 - visualize 105
- tokenize_text 71
- Transforms
 - add_column 49
 - add_unique_id_column 50
 - clamp_values 51
 - Custom 20
 - extract_text_features 53
 - filter 54
 - horizontalize 55
 - ingest_text 57
 - ingest_url 58
 - inner_join 59
 - left_outer_join 60
 - normalize 61
 - remove_columns 63
 - right_outer_join 64
 - set_as_categorical_number 65
 - set_as_categorical_text 66
 - set_as_id 68
 - split 69
 - tokenize_self_contained_text 71
 - vectorize_text 72
- V
- vectorize_text 72
- visualize 105
- visualize_pdp 92

